



OMSCS6250

Computer Networks

Router Design and Algorithms Part I



With Instructor Dr. Maria Konte

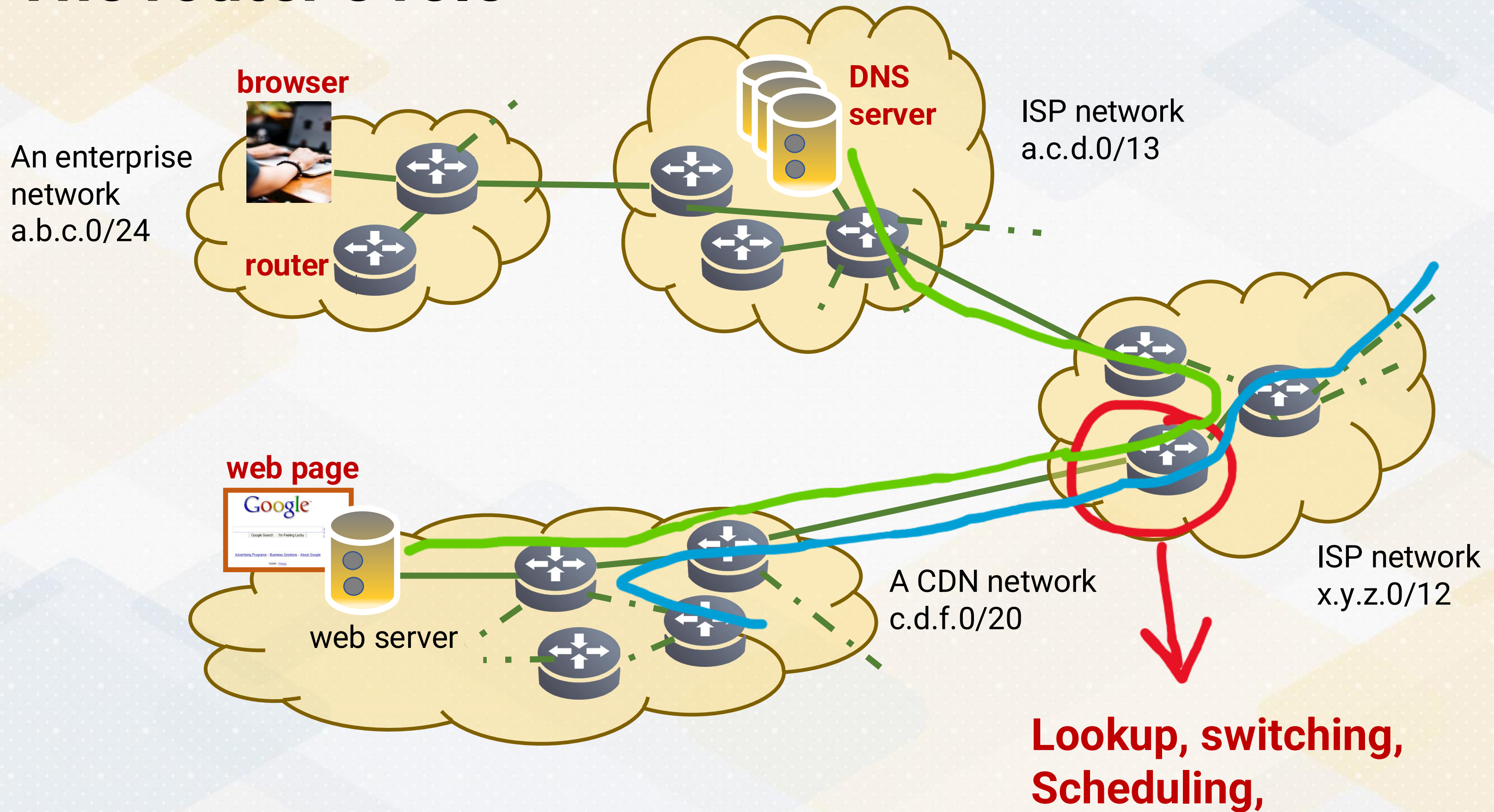
Router architecture, control vs data plane, switching,
scheduling, packet classification, traffic
shaping/policing

Learning Objectives

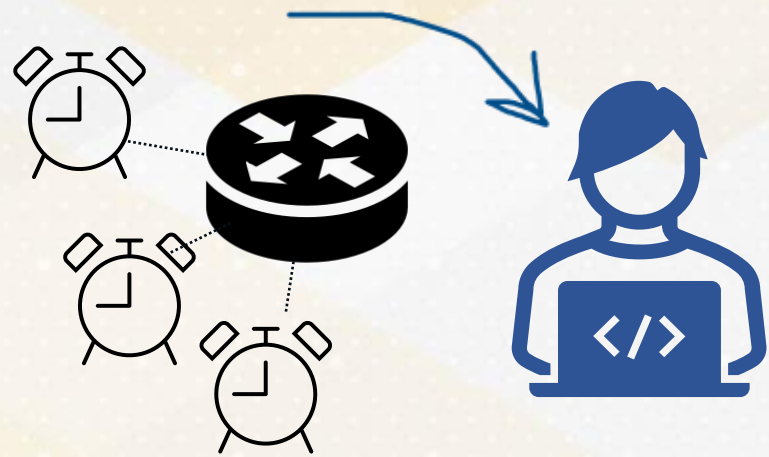
- **Explain what's inside a router and how a packet moves through it** (router components; control plane vs data plane; lookup → switching → queuing/scheduling → output).
- **Compare how routers move packets internally, and why traffic jams happen** (memory/bus/crossbar switching; contention/buffering; delay, loss, throughput limits).
- **Explain how routers choose the “best match” route quickly** (longest-prefix matching; tries—unibit/multibit; prefix expansion; LPM efficiency).
- **Explain how routers share link capacity fairly and smooth bursty traffic** (HOL blocking; VOQ + crossbar scheduling; policing vs shaping; token bucket/leaky bucket).
- **Roadmap**: Router architecture → data vs control plane → lookup & LPM (tries) → switching fabrics (memory/bus/crossbar) → queuing, scheduling & HOL/VOQ → classification + QoS → traffic shaping/policing (token & leaky bucket)

Reviewing the Internet's Protocol Stack

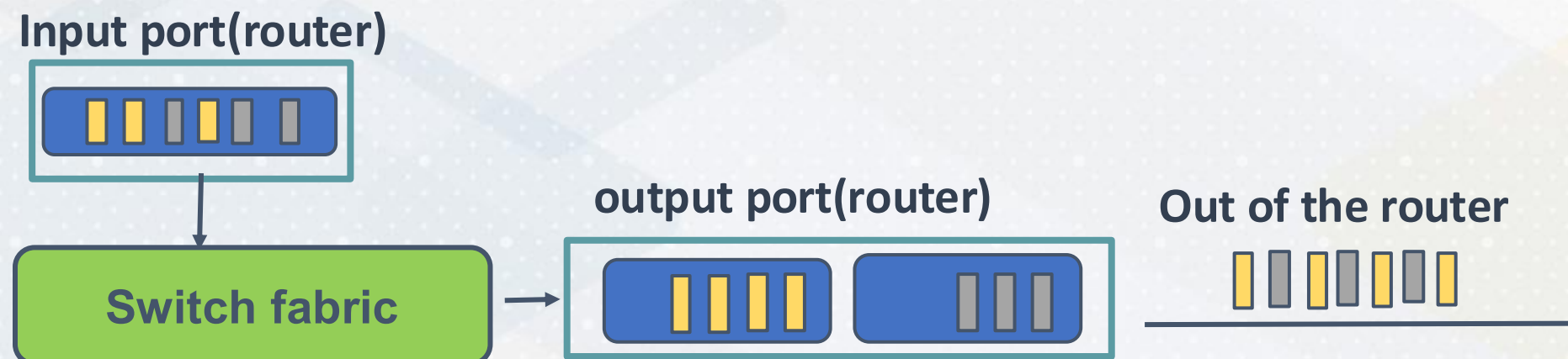
The router's role



The Router's Challenges

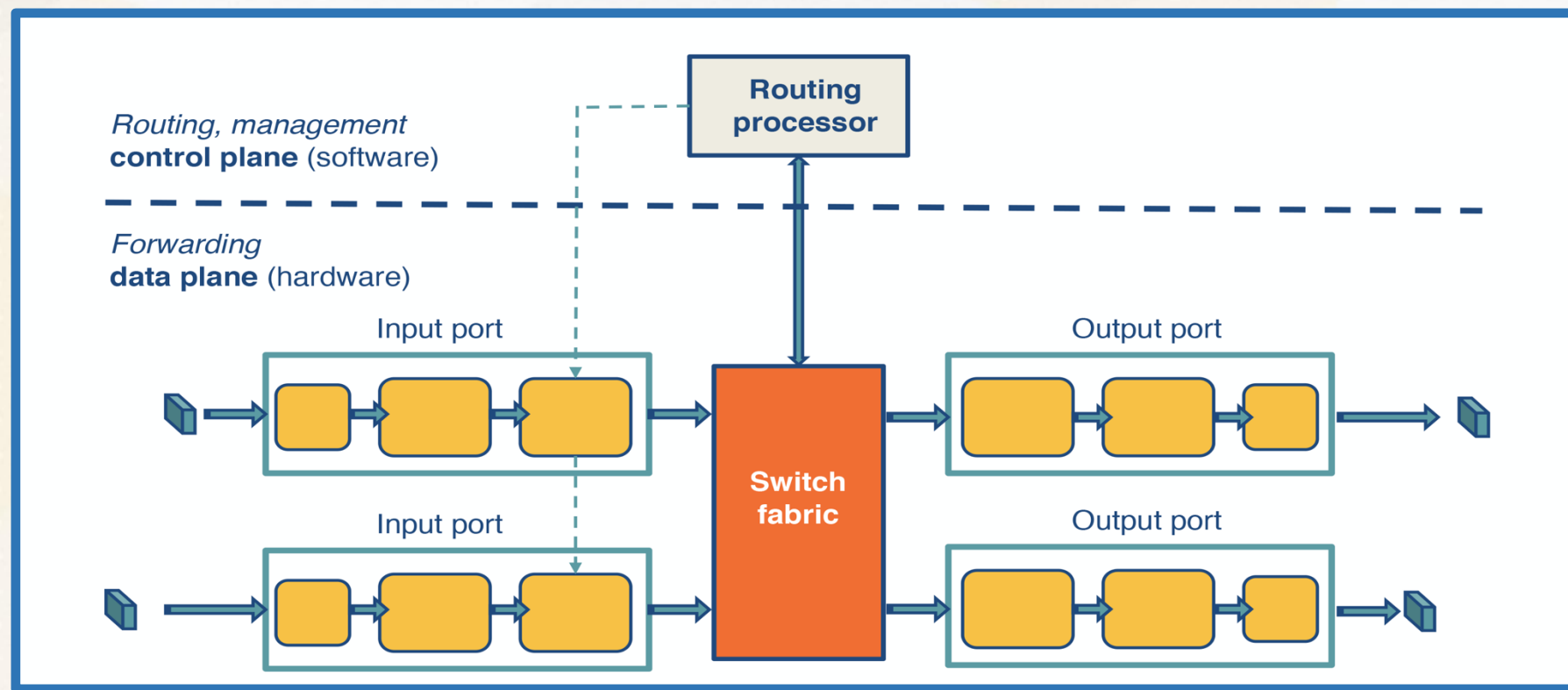


- **Throughput bottlenecks**: internal fabric/memory must keep up with link speeds
- **Queuing effects**: many packets wanting the same output at once creates delay/loss and **HOL blocking**
- **Buffer tradeoff**: buffers absorb bursts but can add large latency if too big
- **Congestion signaling**: drop/mark policy matters
- **Scheduling + fairness**: decide who goes next; balance latency, throughput, and starvation
- **Operations + security**: measurement/telemetry and filtering/policy enforcement add complexity



The Router's Role and Functionalities (1/2)

Control vs Data Plane

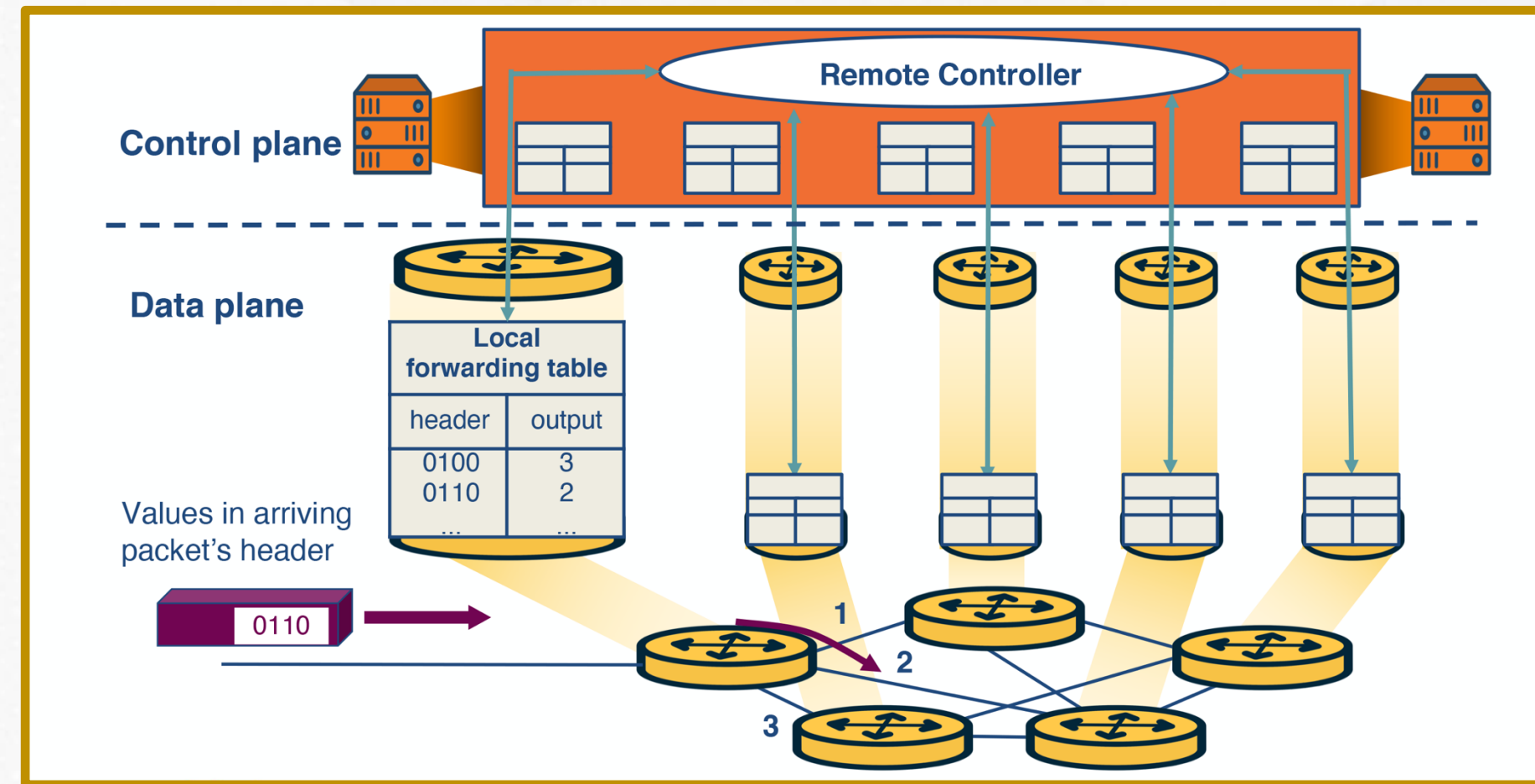


- **Two planes:** control plane (software) computes/installs forwarding state; data plane (hardware) forwards packets at line rate.
- **Input ports:** terminate physical + link layer, then do a **forwarding-table lookup** to pick the right output port; control packets may be sent to the routing processor.
- **Switching fabric:** the internal “network inside the router” that moves packets from inputs to outputs.
- **Output ports:** buffer packets from the fabric and transmit them on the outgoing link (link + physical functions).

The Router's Role and Functionalities (2/2)

Control vs Data Plane

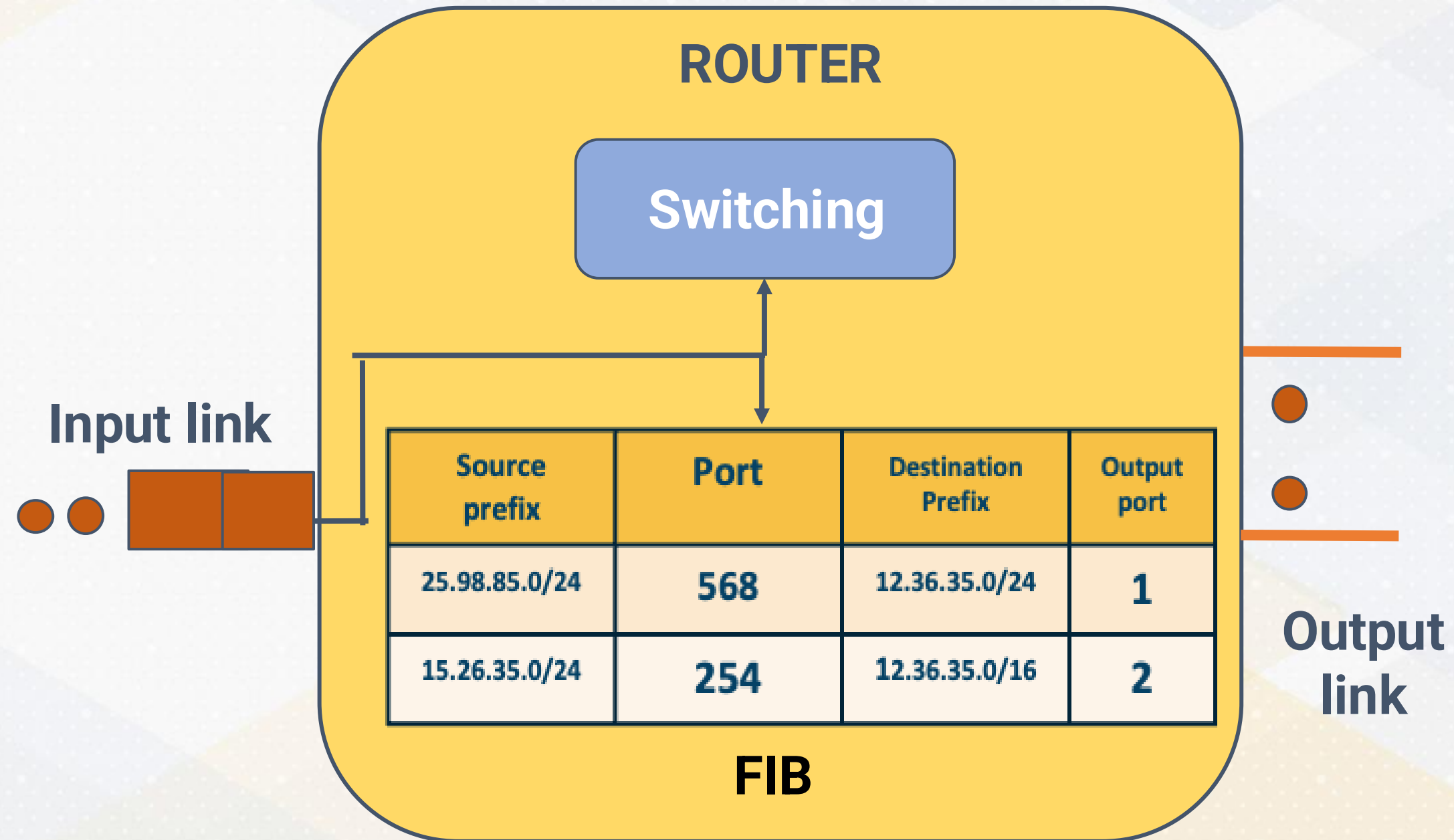
- **Control plane = intelligence**: participate in routing protocols, can also be decoupled from the router and be implemented as a (logically) centralized controller that computes policy/routes and **installs rules** on devices.
- **Data plane = fast path**: each router/switch uses a **local forwarding table** to map header bits → output port at line rate.
- **Separation**: controller decides *what* should happen; devices perform *per-packet* actions quickly and repeatedly.
- **Key benefit**: simpler devices + easier network-wide control/updates.



What Happens When a Packet Arrives at an Input Port?

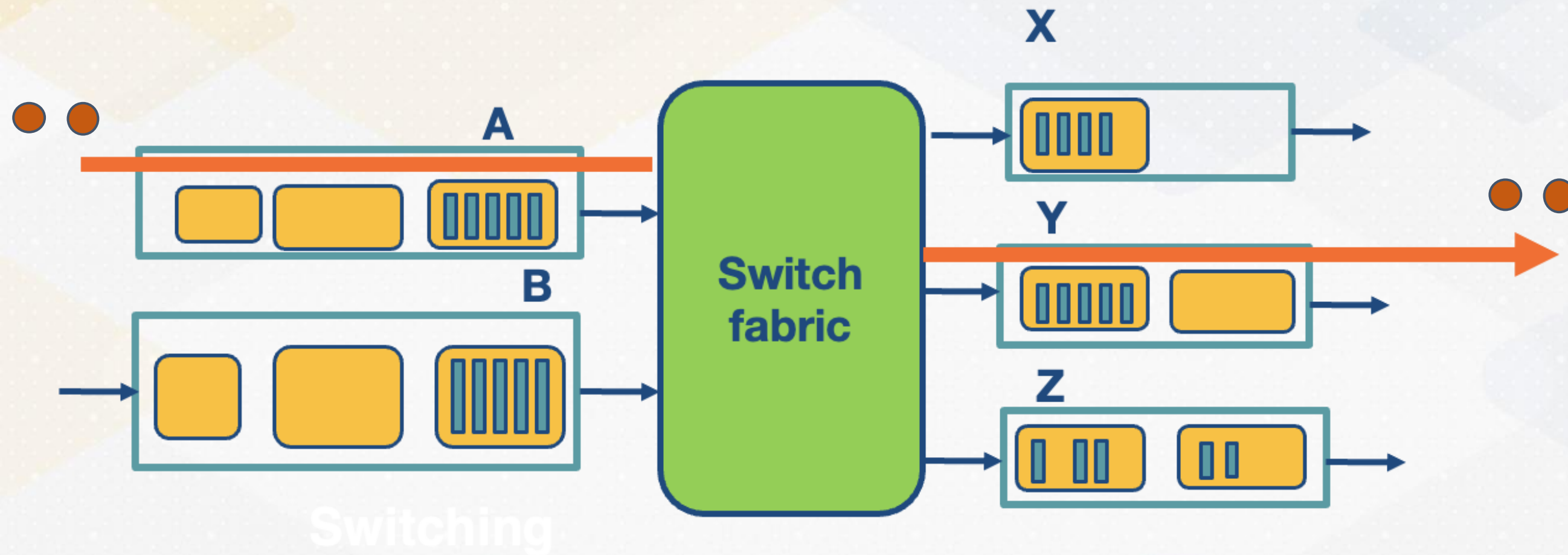
Longest Prefix Matching

- When a packet arrives, the router does a **forwarding lookup** in the **FIB** (Forwarding Information Base).
- The lookup matches the packet's **destination IP prefix** (typically **longest-prefix match**) to choose an **output port**.
- This is a **data-plane** operation: fast, per-packet, designed to run at line rate.
- After lookup, the packet is sent into the **switching fabric** toward the chosen output link.



What Happens When a Packet Arrives at an Input Port?

Switching

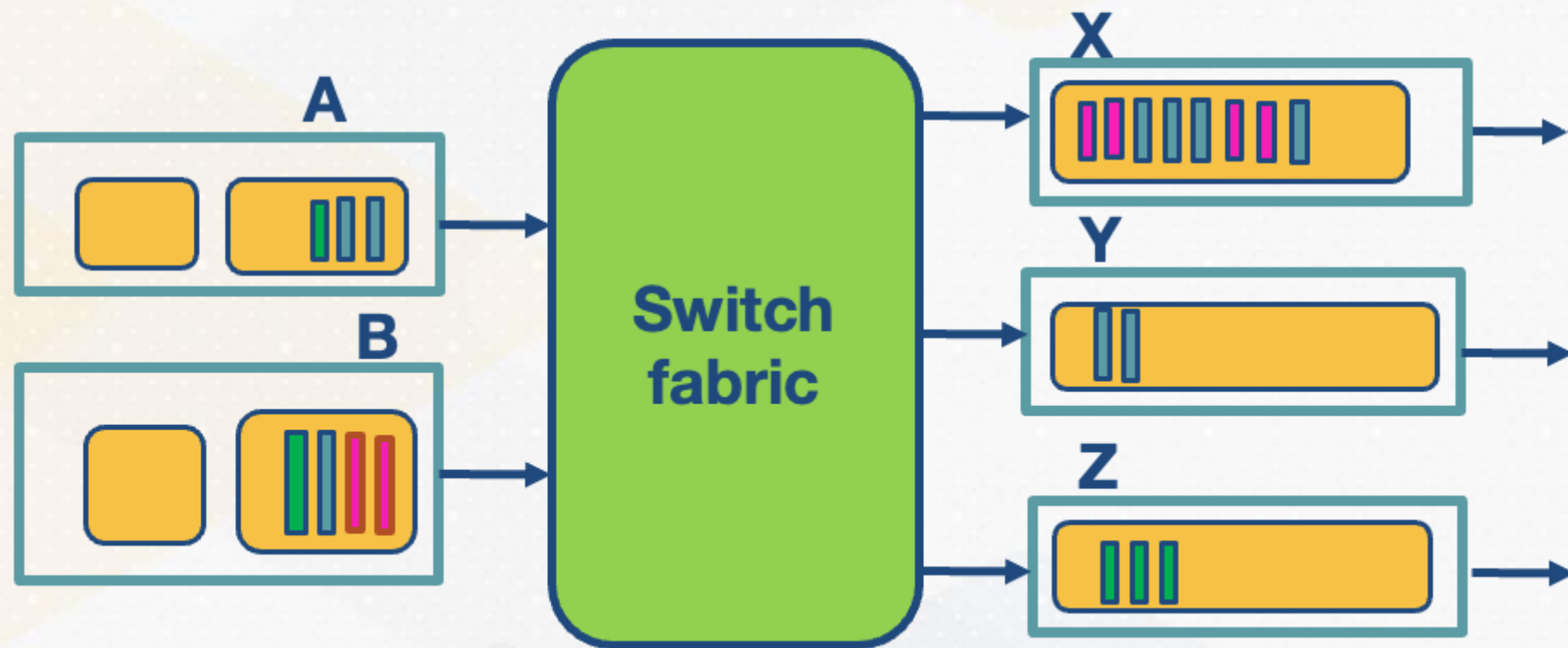


- After **lookup**, the packet must cross the **switching fabric** to reach the chosen output port.
- The switching fabric is the router's internal "backplane" that connects **many inputs to many outputs**.

- If multiple inputs want the **same output** at the same time, the router must **arbitrate** → packets may wait (internal contention).
- **Goal**: move packets fast enough that internal switching doesn't become the bottleneck.

What Happens When a Packet Arrives at an Input Port?

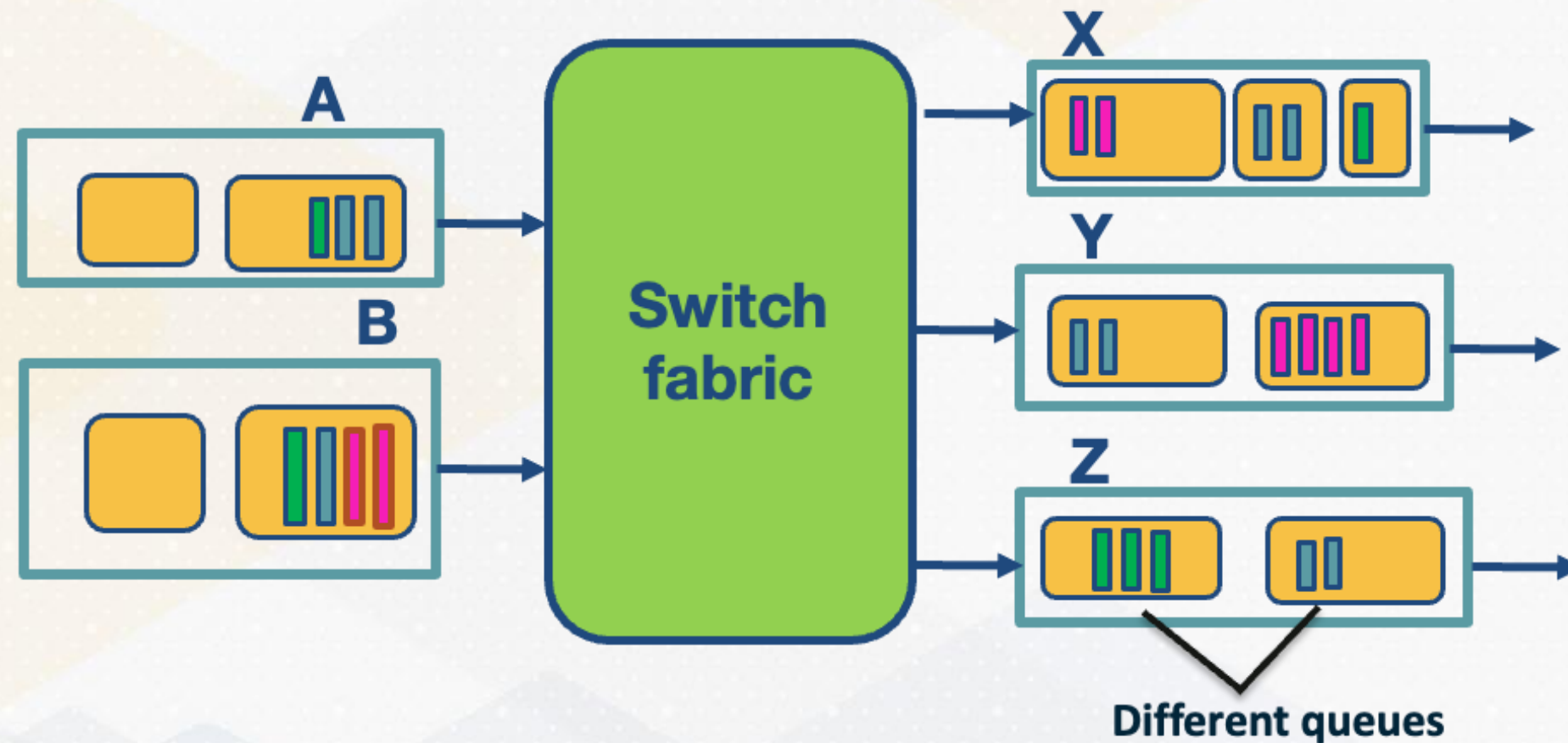
Queuing



- After switching, packets may need to **wait in buffers** because outputs can transmit only so fast.
- Queues form when **arrival rate** > **service rate** (even briefly, during bursts).
- Queuing adds **delay**; if buffers fill, packets are **dropped** (loss).
- Routers may use **different queues** per output (by class/flow) before scheduling transmissions.

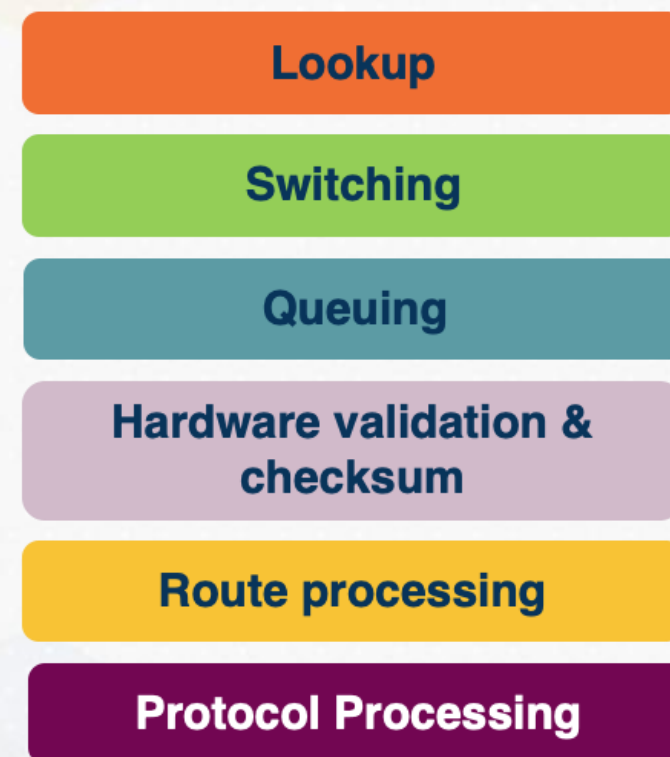
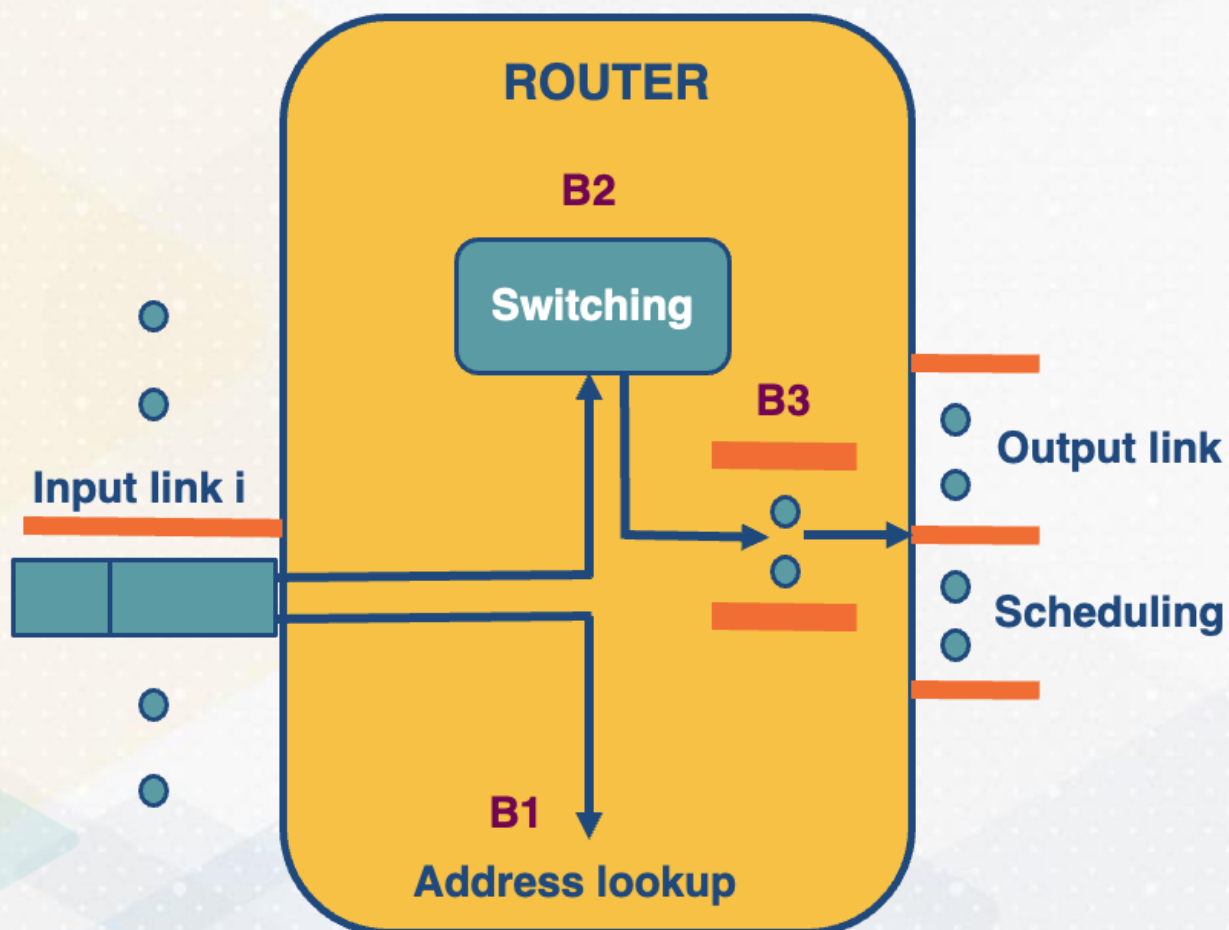
What Happens When a Packet Arrives at an Input Port?

Scheduling



- Routers often keep **multiple queues per output** (by traffic class or flow), not just one FIFO line.
- This enables **QoS + fairness**: protect latency-sensitive traffic and prevent one heavy flow from dominating.
- A **scheduler** chooses the next packet across queues (e.g., priority vs weighted/fair scheduling), trading off **latency, throughput, and starvation risk**.

The Router's Architecture

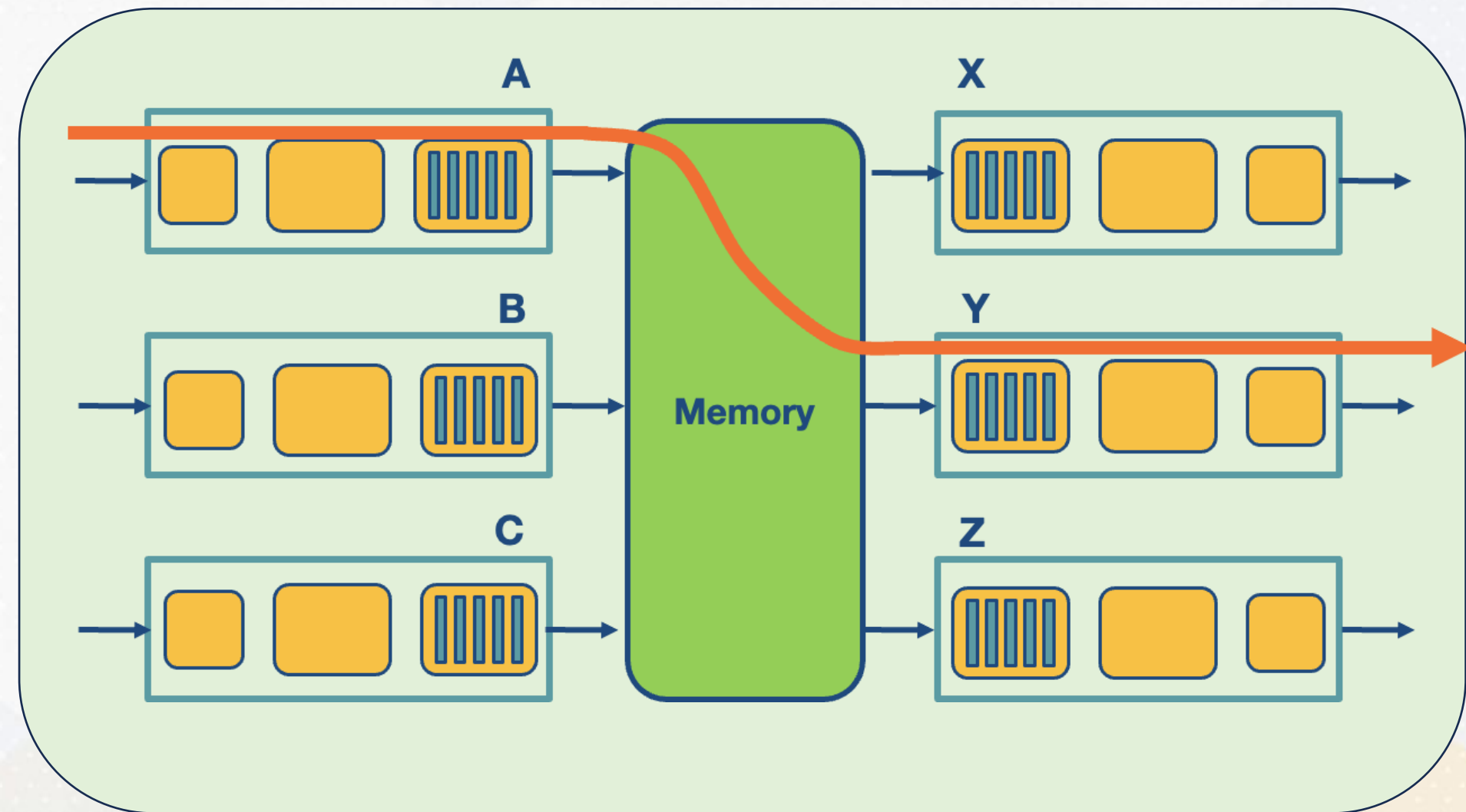
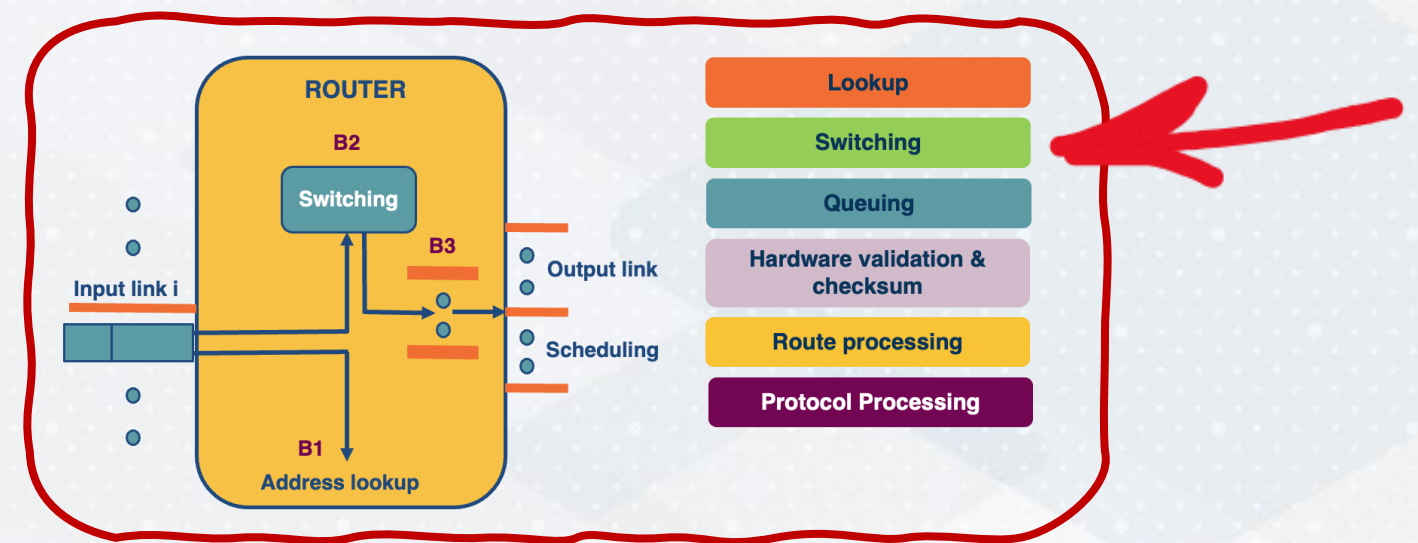


- Data-plane pipeline: **lookup** → **switching** → **queuing** → **scheduling** → **output link**.
- **Hardware validation + checksum** happen inline to keep forwarding correct at high speed.
- Control-plane side: **route processing + protocol processing** maintain the tables the data plane uses.
- Key idea: **fast per-packet forwarding** separated from **slower control/management tasks**.

Switching

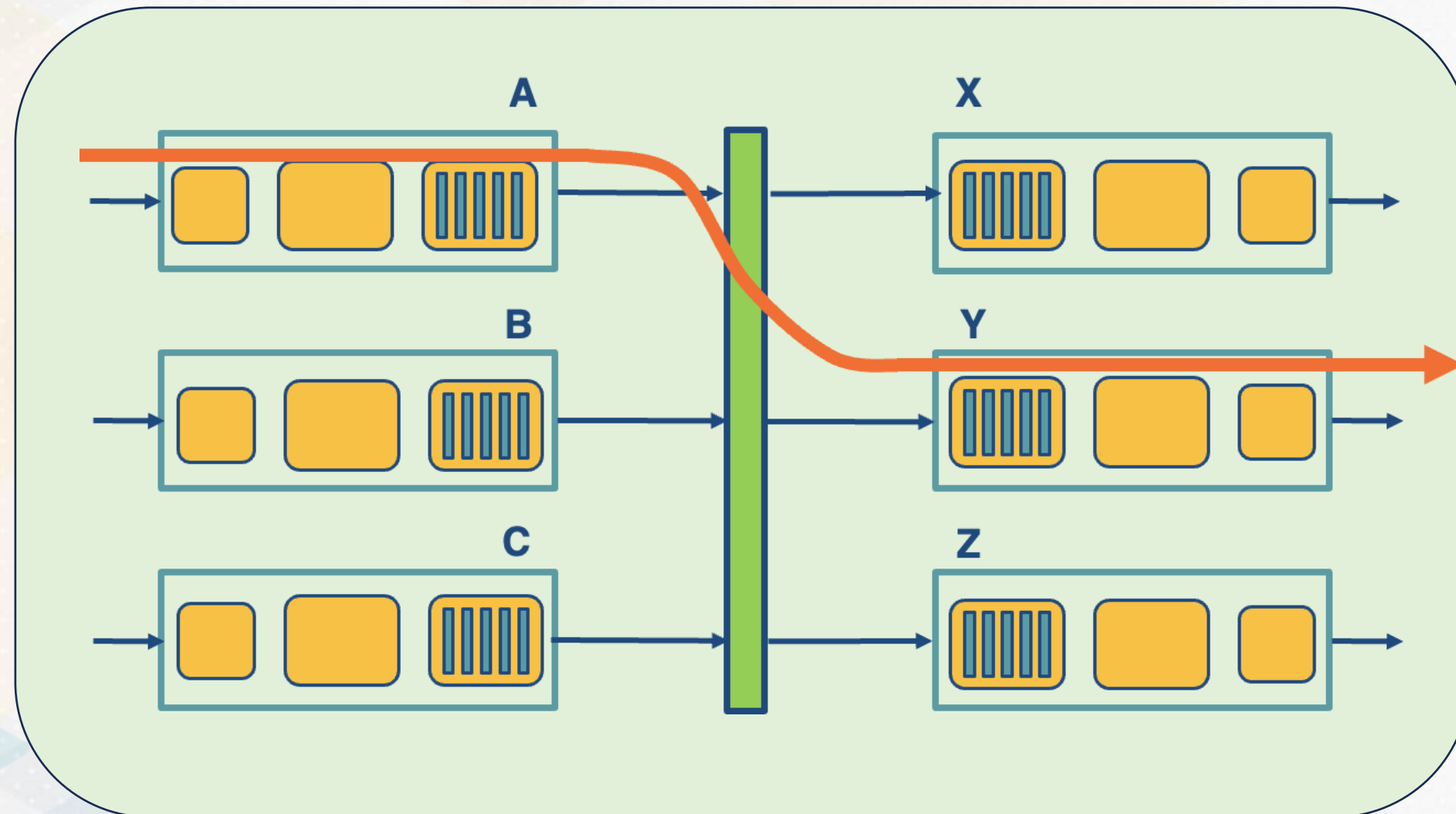
Switching via memory

- Packet is **copied into shared router memory**, then **copied out** to the chosen output port.
- The **memory bandwidth** limits throughput (often needs **$\sim 2\times$ line rate**: one write + one read per packet).
- Simple design (like early routers), but **doesn't scale** well to many high-speed ports.



Switching

Switching via bus

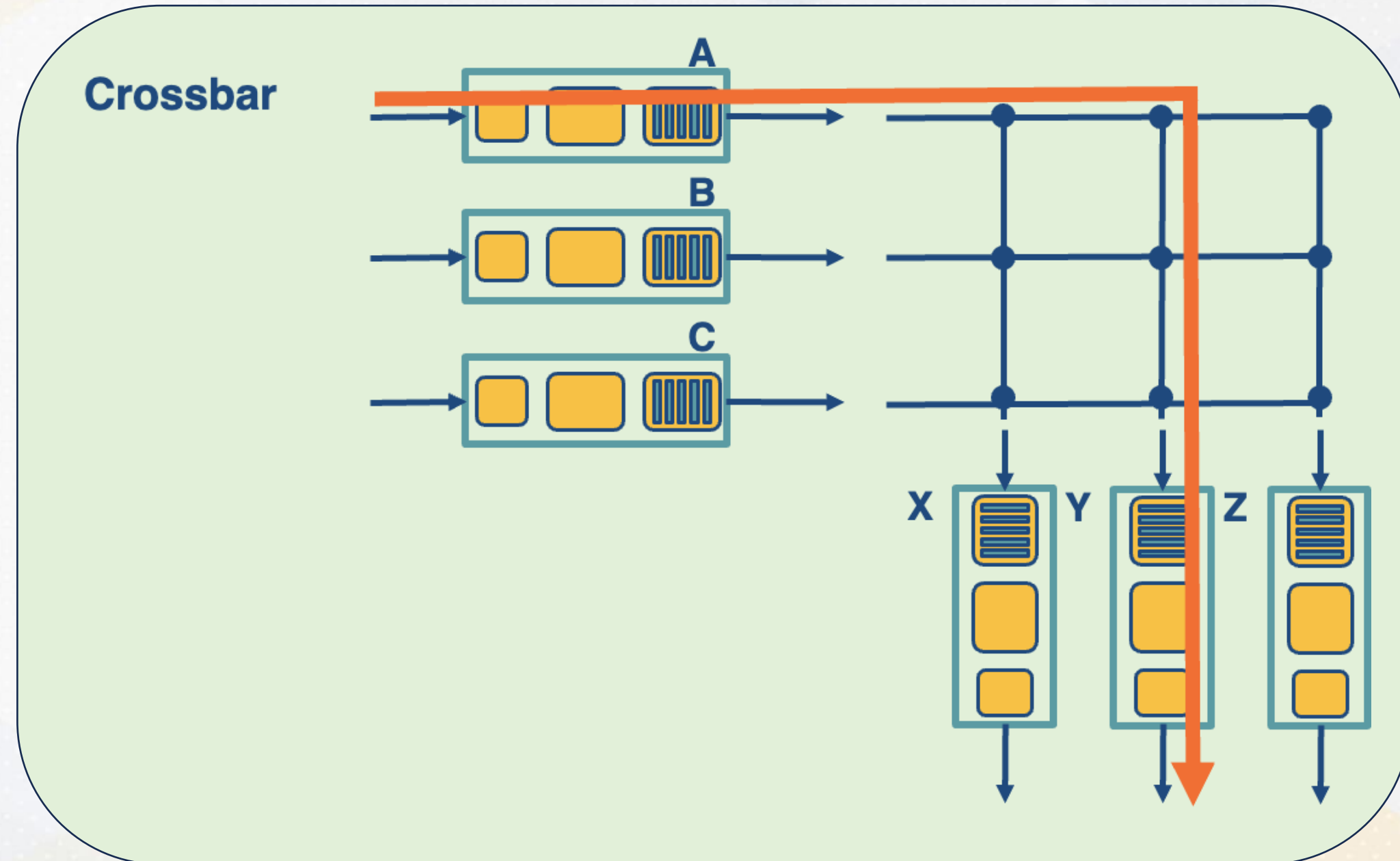


- **A shared bus** is a single internal pathway connecting all inputs to all outputs.
- Packets **take turns** using the bus (arbitration), so **only one packet can cross at a time**.
- Simple and low-cost, but the **bus becomes the bottleneck** as traffic and port speeds grow.

Switching

Switching via crossbar switch

- A **crossbar** creates configurable connections between **inputs and outputs**, enabling **parallel transfers**.
- Multiple packets can cross **simultaneously** as long as they go to **different outputs**.
- If multiple inputs target the **same output**, you still need **arbitration/queuing** at that output.





OMSCS6250

Computer Networks

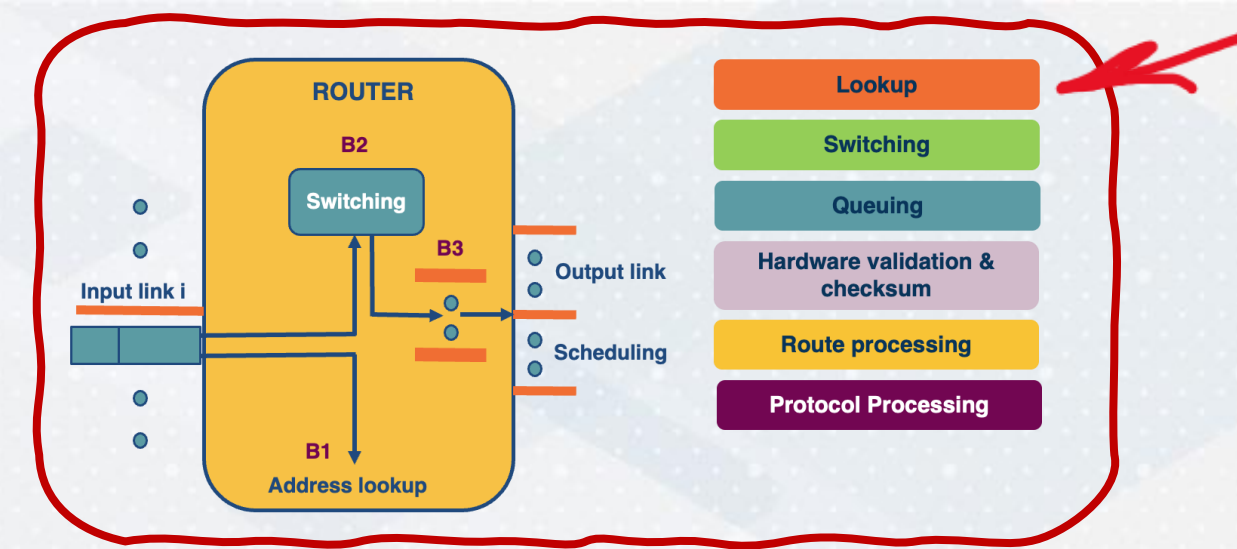
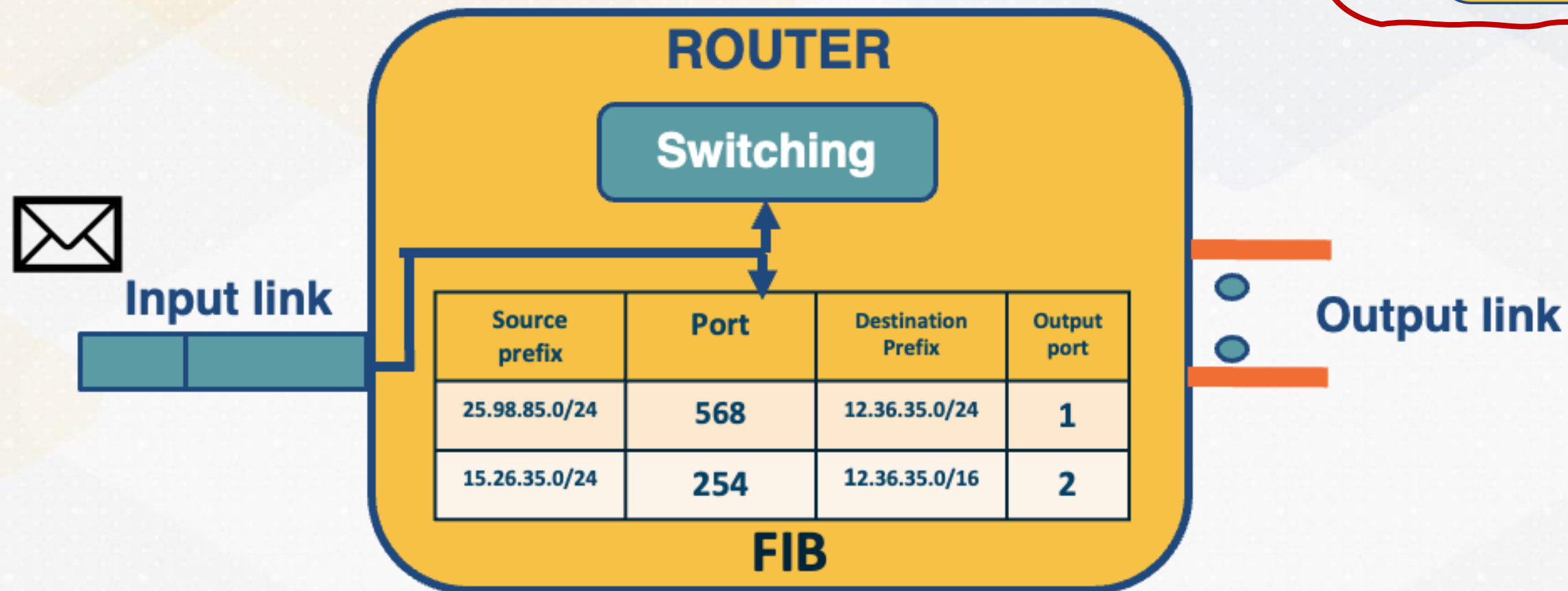
Router Design and Algorithms Part II



With Instructor Dr. Maria Konte

Router architecture, control vs data plane, switching,
scheduling, packet classification, traffic
shaping/policing

Longest Prefix Matching

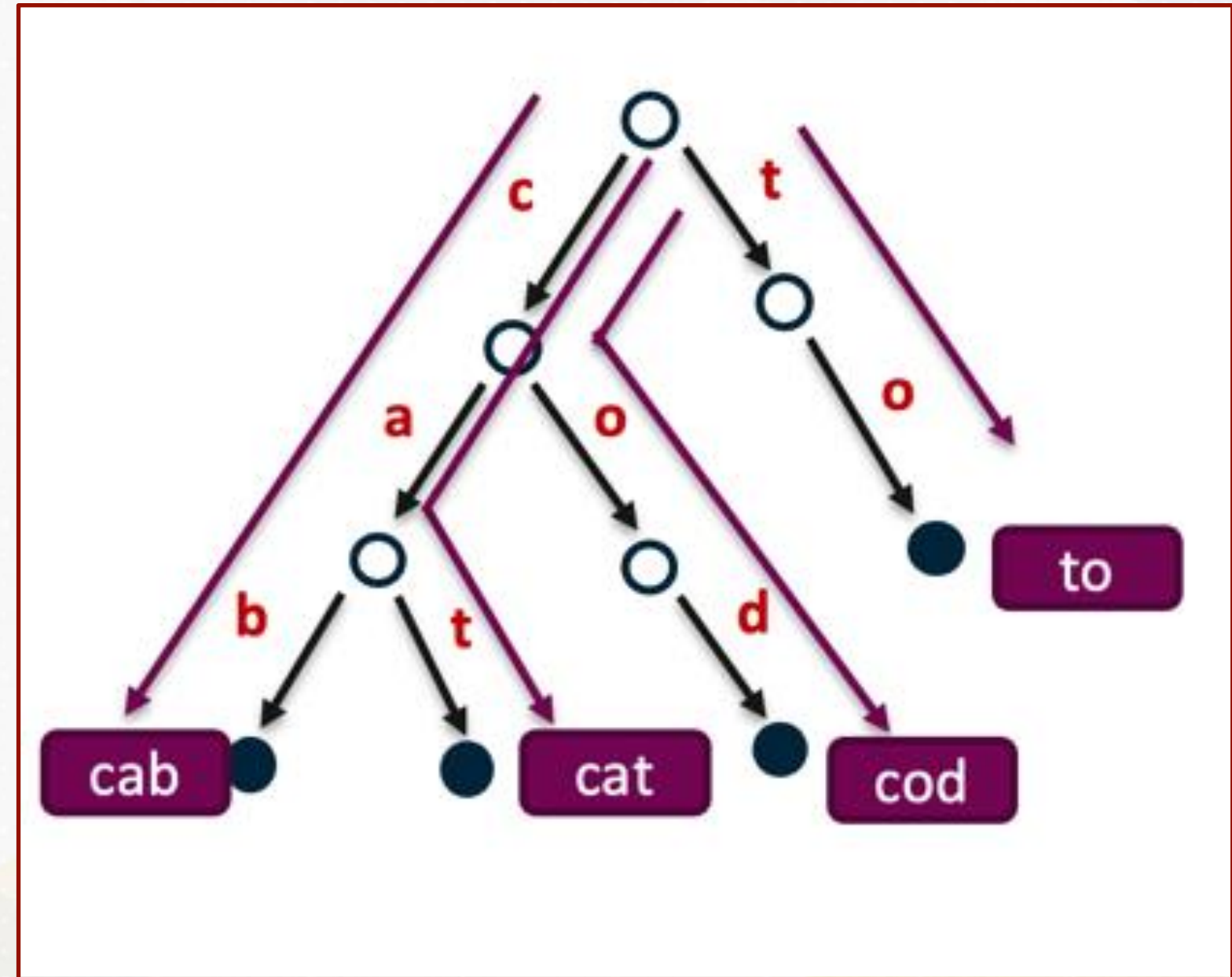


- Router forwards by looking up the destination IP in the **FIB** using **prefixes** (CIDR blocks).
- If multiple prefixes match, choose the **longest prefix** (most specific match).
- This makes forwarding **correct and scalable**: default routes for “everything else,” specific routes for subnets.

Longest Prefix Matching

Unibit Tries (1/2)

- A **trie** is a tree that stores keys by sharing common prefixes.
- In networking, we **store IP prefixes** in a trie so lookup follows the address **one step at a time**.
- **Each step in the trie consumes the next bit of the destination address** and takes us left or right. As we walk, we may pass nodes that represent valid prefixes -> these are routing table entries.
- **Longest-prefix match** = while walking down, remember the **last prefix** that matched.

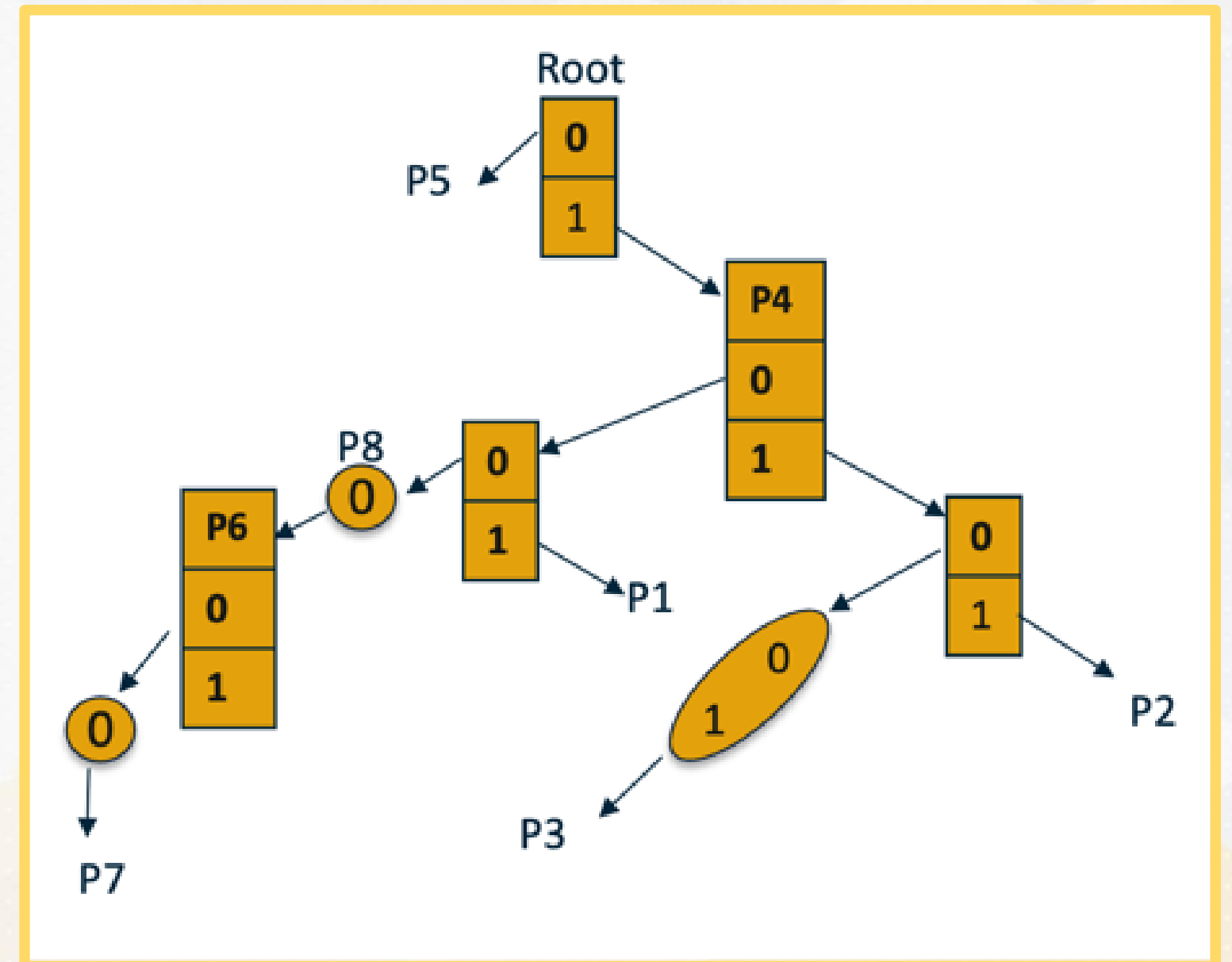


Longest Prefix Matching

Unibit Tries (2/2)

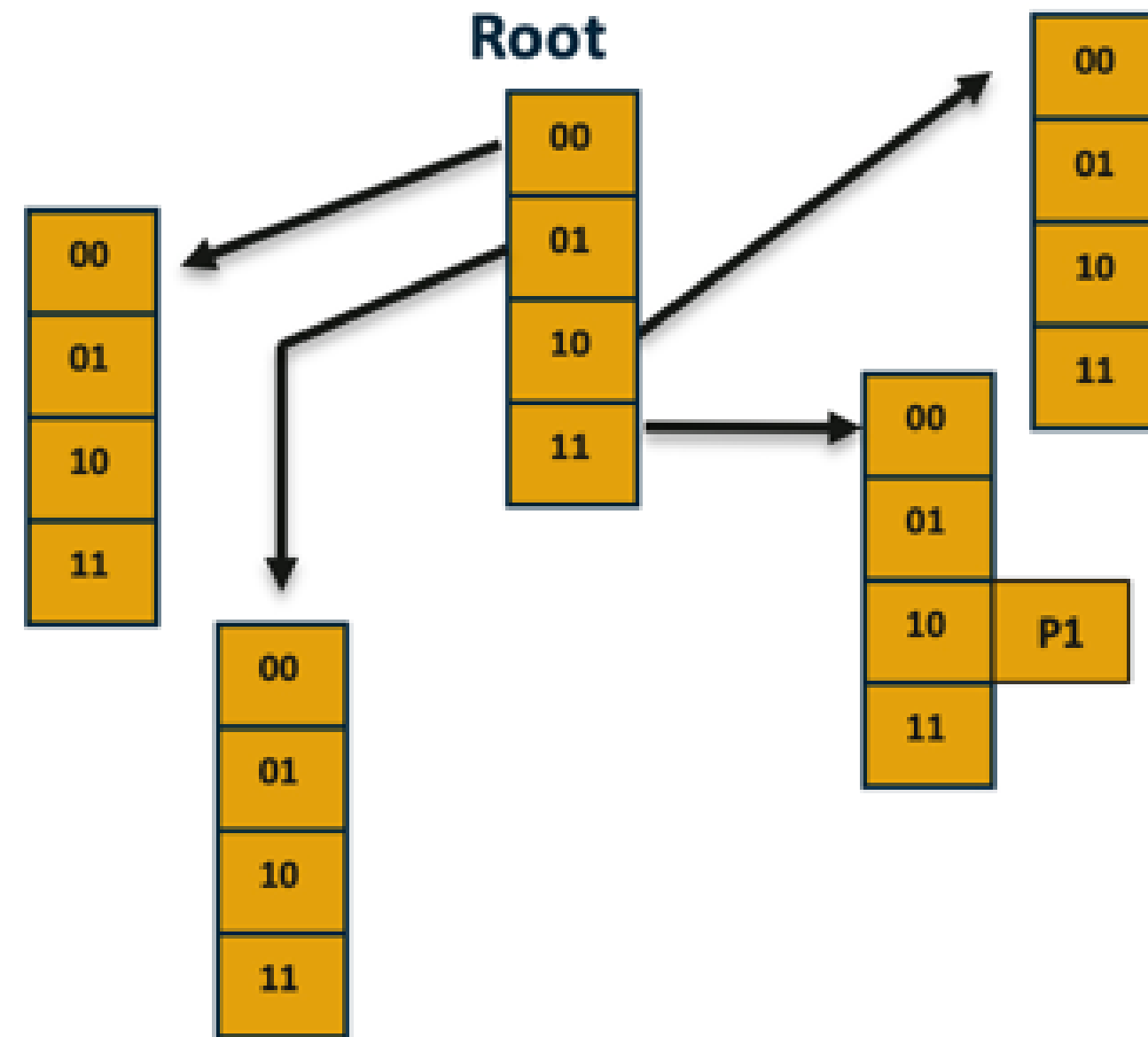
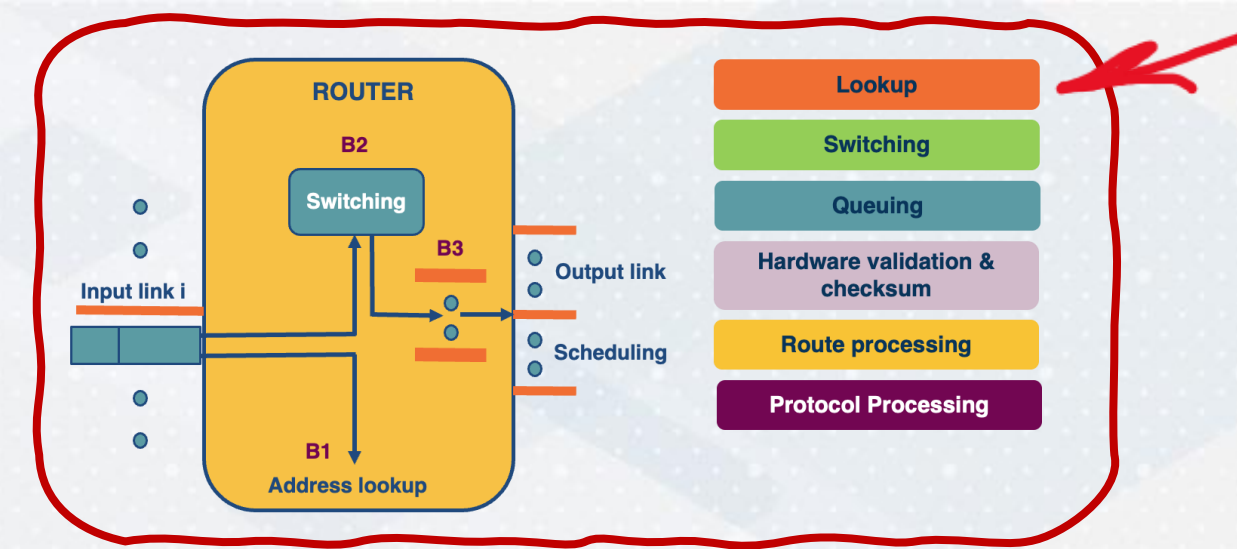
- A **unibit trie** branches on **one bit per level** (0 vs 1).
- Lookup time is **$O(W)$** in the worst case (W = address length, e.g., up to 32 for IPv4), because you may check many levels.
- Unibit tries are the simplest way to implement longest-prefix match, **a baseline** that more optimized lookup schemes improve on.

P1 = 101*
P2 = 111*
P3 = 11001*
P4 = 1*
P5 = 0*
P6 = 1000*
P7 = 100000*
P8 = 100*
P9 = 110



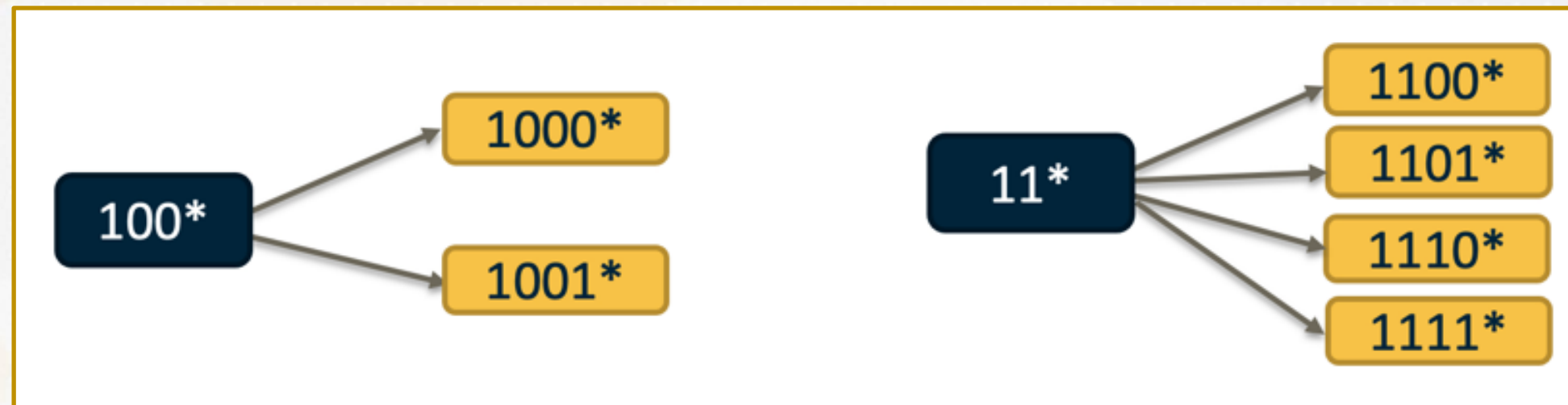
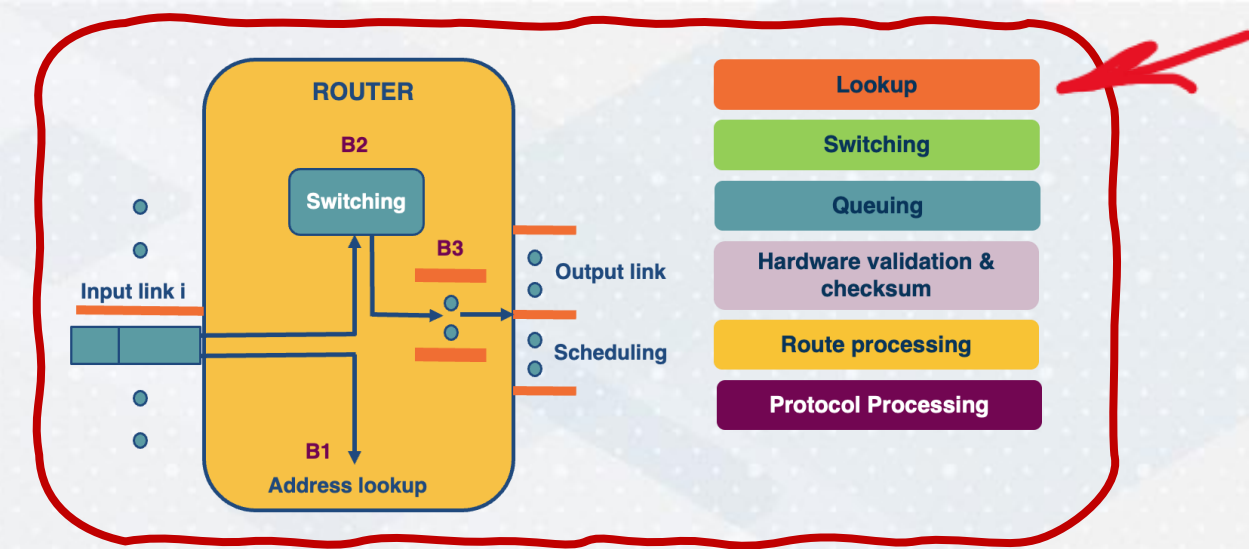
Longest Prefix Matching Multibit Tries

- Instead of 1 bit per level, a **multibit trie** uses **a stride** (here, 2 bits), so each node has up to **4 children: 00, 01, 10, 11**.
- Lookup becomes **fewer steps** (consume 2 bits at a time), which speeds up longest-prefix matching.
- **Tradeoff:** nodes can be **larger/sparser**, so you may spend more memory to get faster lookup



Longest Prefix Matching

Prefix Expansion (1/3)

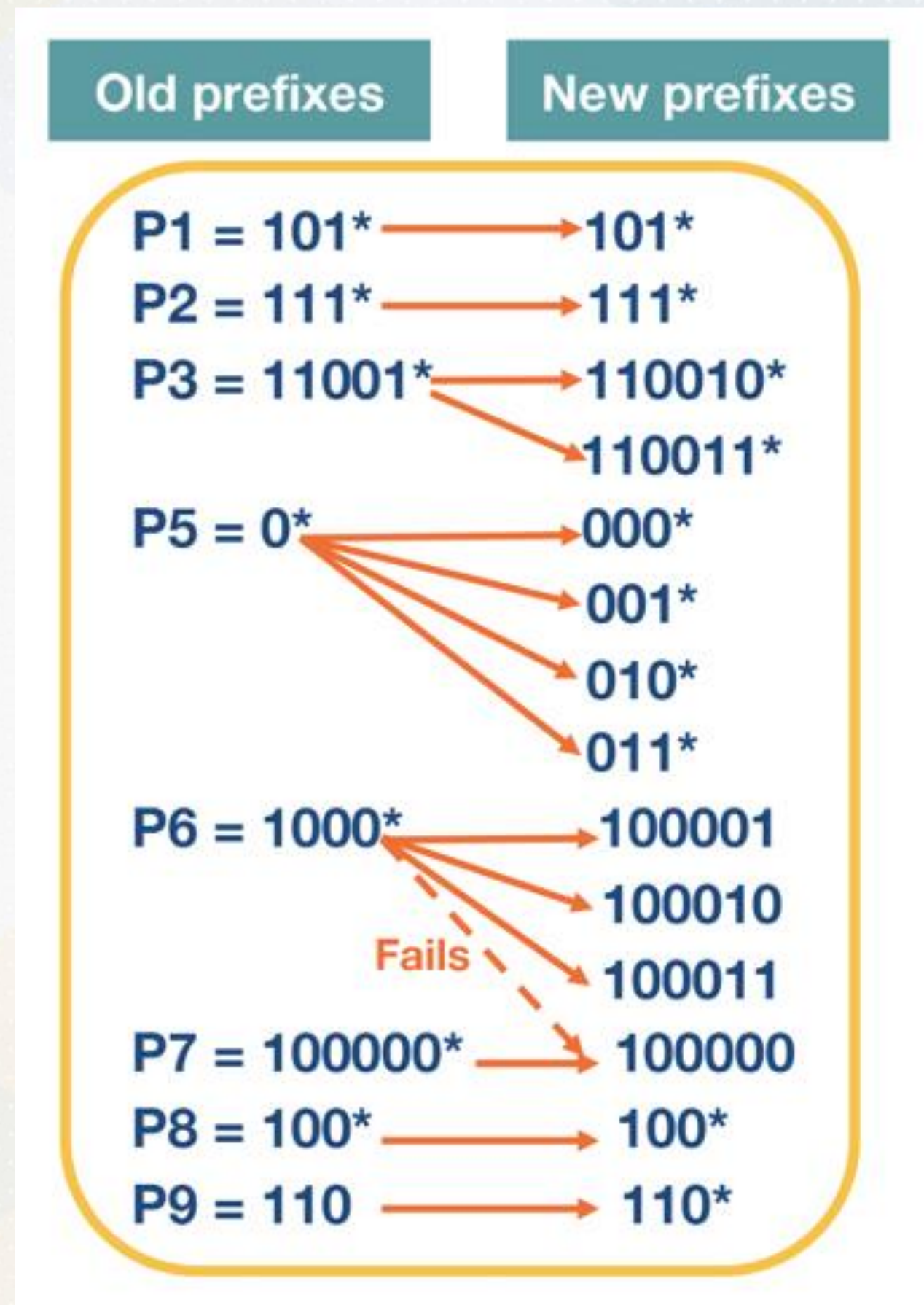


- With multibit tries, prefixes don't always align with the stride, so we use prefix expansion to make them fit.
- **Expansion replaces a shorter prefix with a set of longer, equivalent prefixes (e.g., $11^* \rightarrow 1100^*, 1101^*, 1110^*, 1111^*$).**
- Benefit: enables **fast table indexing per stride**
- Cost: more entries + **more memory**.

Longest Prefix Matching

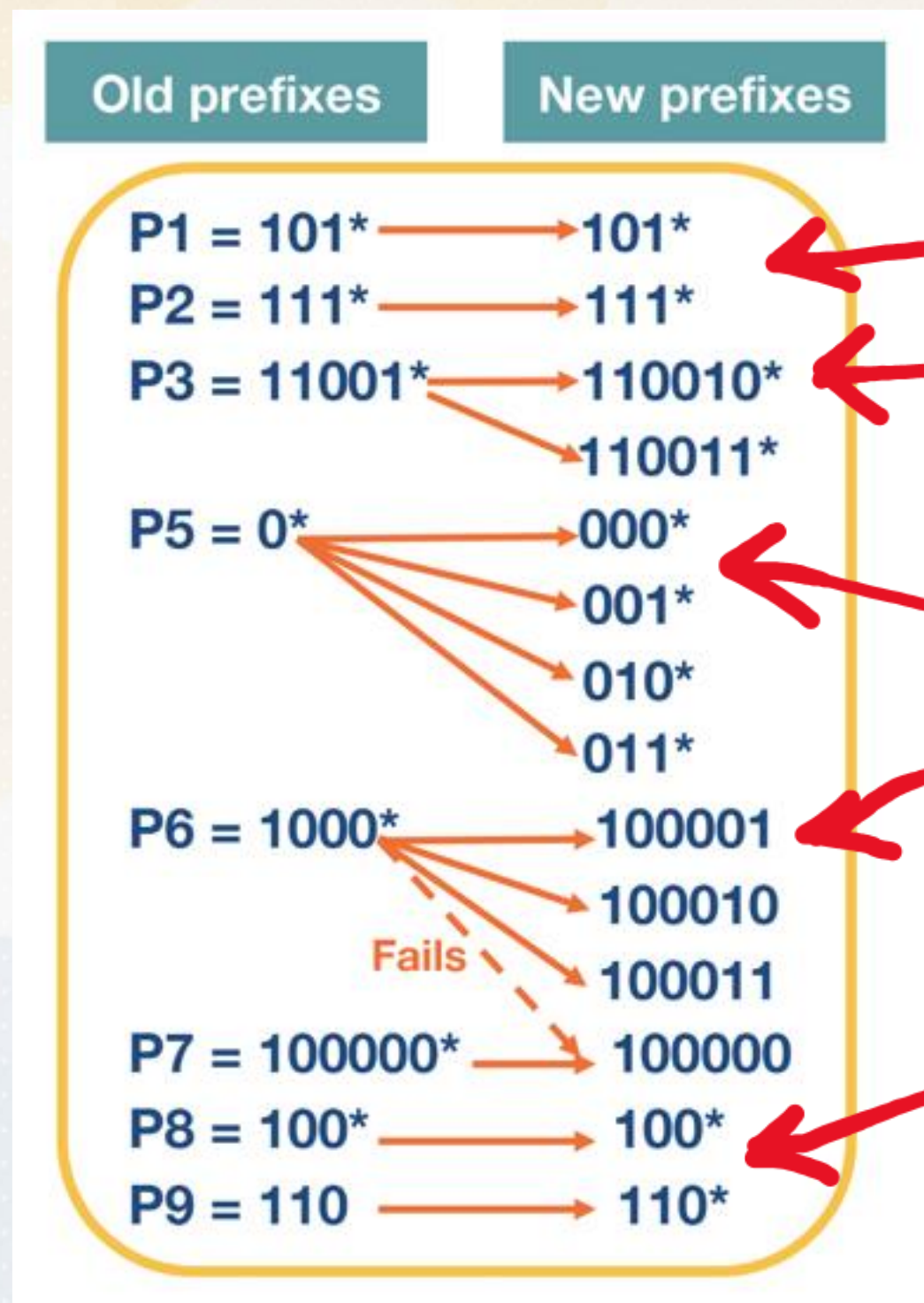
Prefix Expansion (2/3)

- Goal: Expand every prefix to a length that's a **multiple of 3** (for a 3-bit multibit trie).
- If a prefix is too short, append bits and generate **all combinations to reach the next multiple of 3**.
- If an expanded prefix **collides** with an existing entry, the expanded copy **is dropped (more-specific prefix wins)**.



Longest Prefix Matching

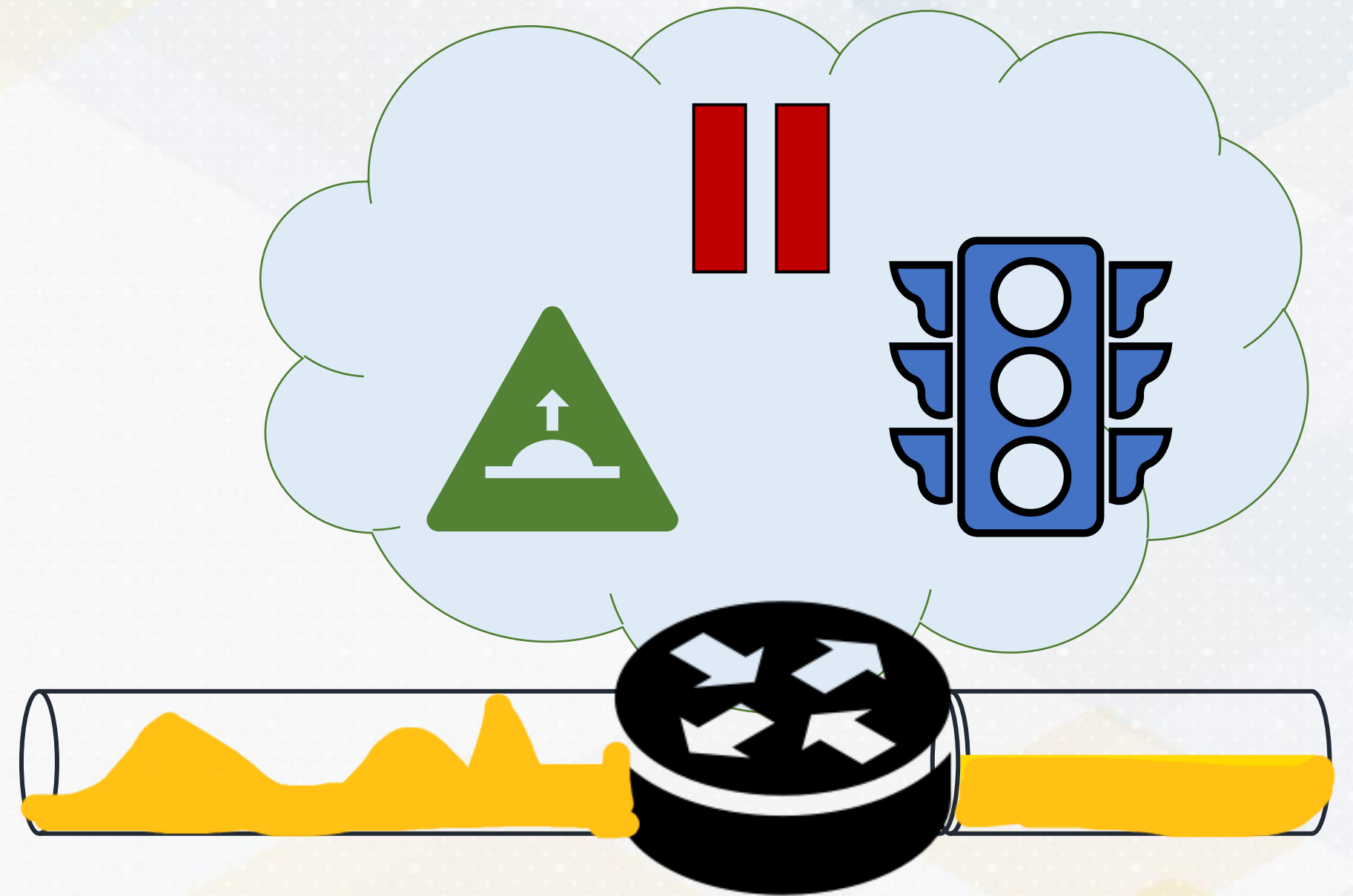
Prefix Expansion (3/3)



- P1 stays (already length 3)
- P2 stays (already length 3)
- P3 → 110010* and 110011*
- P4 = 1* → expands to 100*, 101*, 110*, 111* → all dropped here due to collisions (those already exist)
- P5 = 0* → 000*, 001*, 010*, 011*
- P6 = 1000* → 100000*, 100001*, 100010*, 100011* → but 100000* collides (exists as P7) so drop 100000*
- P8 → no expansion needed
- P9 → no expansion needed

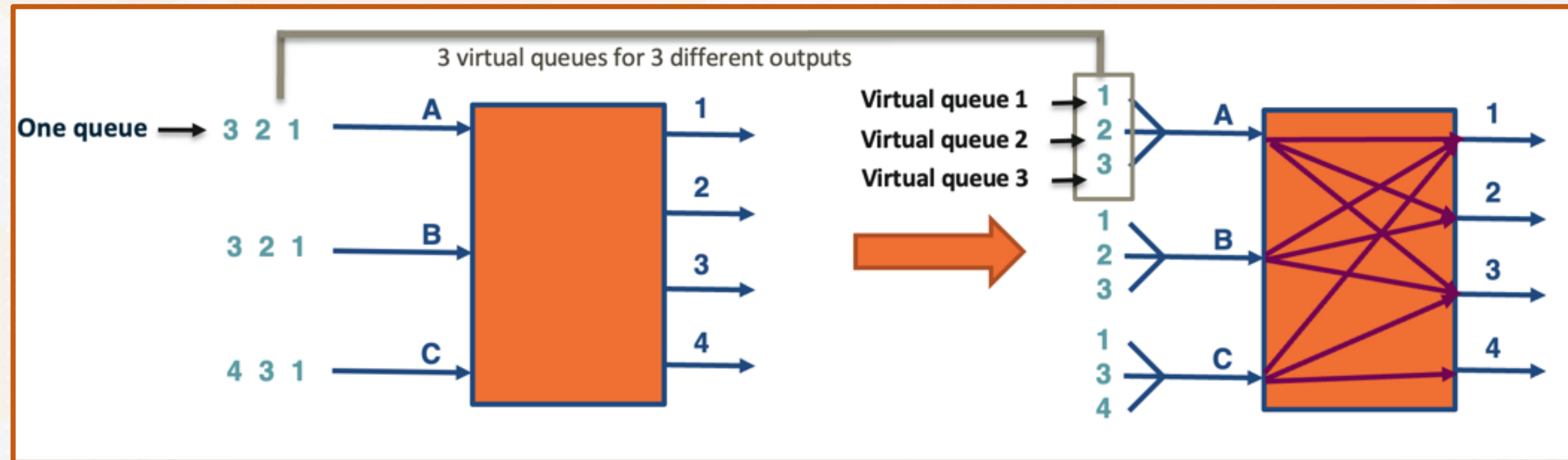
Why scheduling/QoS?

- **Busy routers must schedule in real time**: decide which packet/flow goes next at very small inter-packet times.
- **FIFO + tail drop is simple**: one queue per output, drop when full — fast but can drop important traffic.
- **QoS motivation**: support congestion, protect important flows, and **fairly share** link bandwidth/delay across flows.



Scheduling

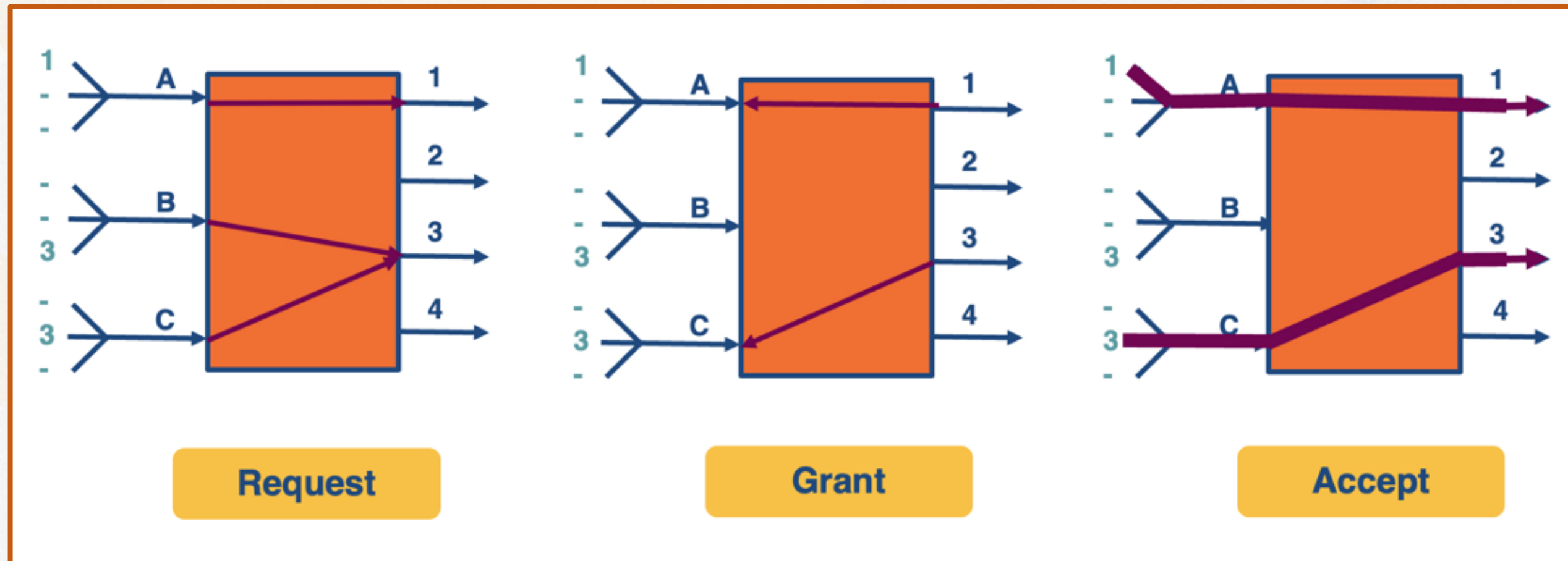
HOL Blocking & Parallel Iterative Matching (1/2)



- **HOL blocking**: with a single FIFO input queue, a packet at the front can block packets behind it even if those behind packets want free outputs.
- **Fix: Virtual Output Queues (VOQ)** – each input keeps **a separate queue per output**, so one congested output doesn't block the rest.
- A crossbar scheduler can use **parallel iterative matching** (request → grant → accept) to pick a conflict-free set of input→output transfers each time slot.

Scheduling

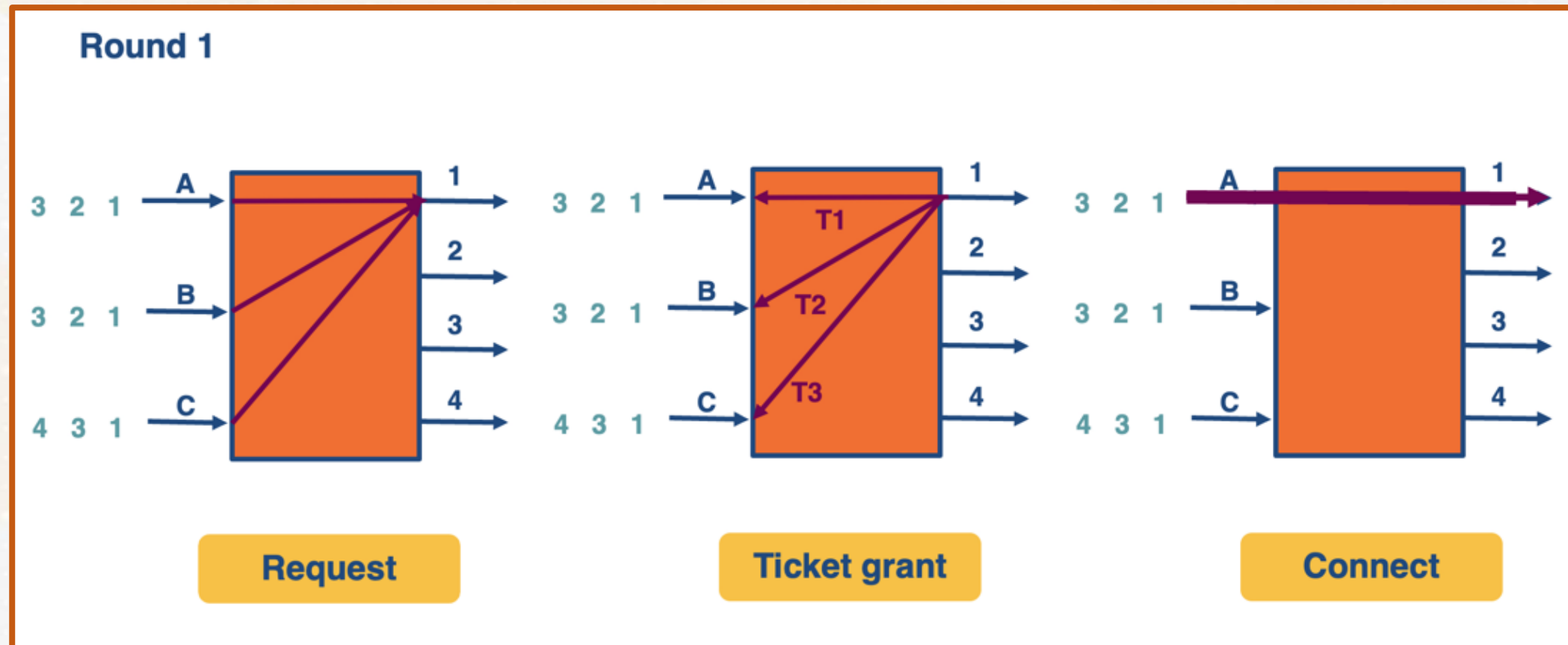
HOL Blocking & Parallel Iterative Matching (2/2)



- **Request:** each input requests the output for the head packet of each VQ.
- **Grant:** each output picks one request to grant (resolving output contention).
- **Accept:** each input accepts at most one grant → forms a **matching** for that time slot.

Scheduling

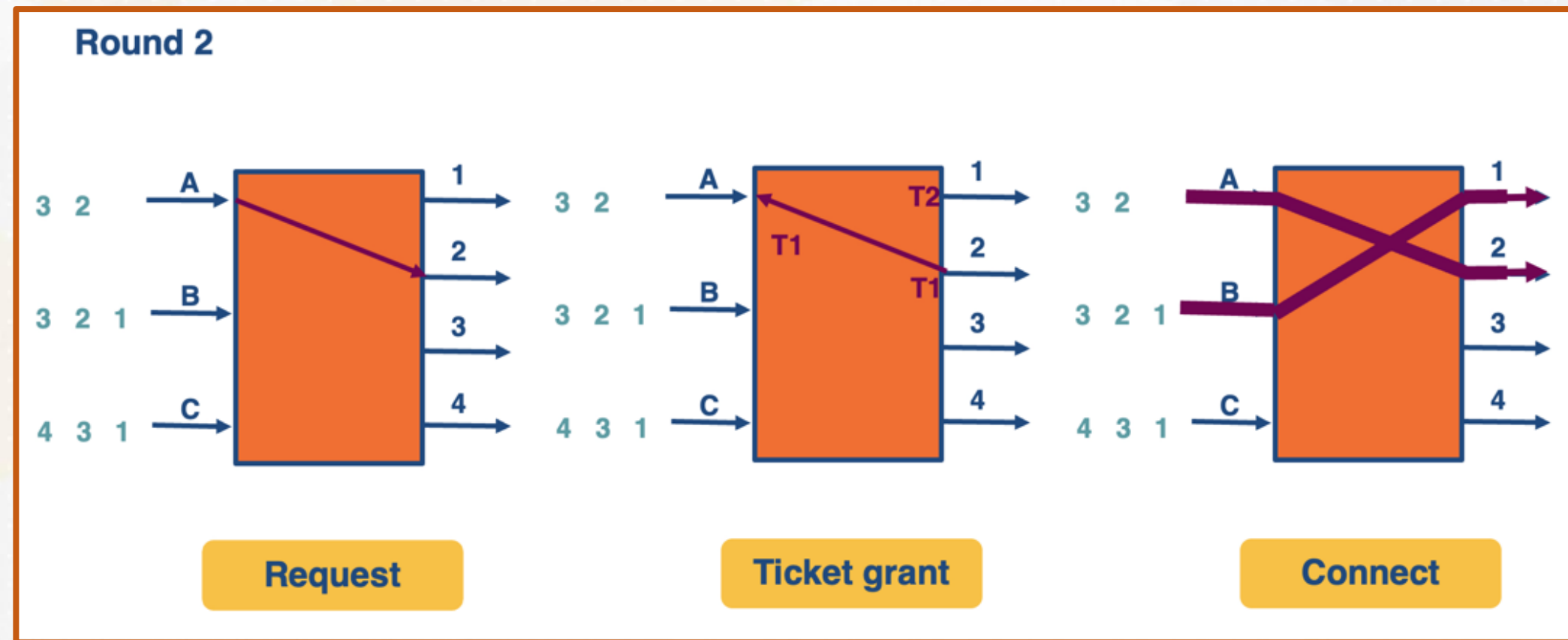
Take a ticket scheduling mechanism (1/3)



- **Gap it fills:** VOQs remove HOL blocking, but they create a new question: *when many inputs request the same output, who gets through this time slot?*
- **High-level goal:** pick a **conflict-free matching** each slot (no input or output used twice) that keeps the crossbar **high utilization and stays fair**.
- **Round 1:** inputs **request**, outputs assign/grant a **ticket-based winner**, then the crossbar **connects** the winning input→output pairs.

Scheduling

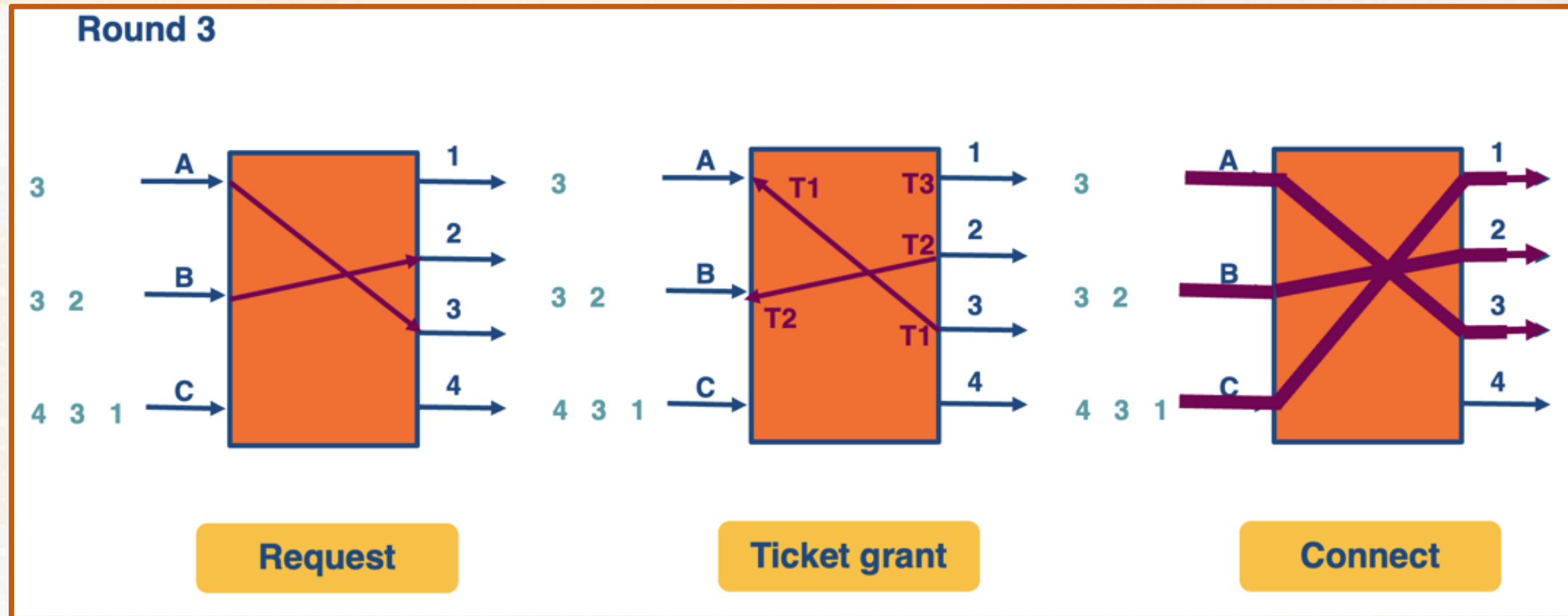
Take a ticket scheduling mechanism (2/3)



- After Round 1, some requests are still **unmatched** because of conflicts.
- Run another round: remaining inputs **re-request**, outputs **grant** based on ticket order again.
- Each round increases the size/quality of the **matching** without heavy computation.

Scheduling

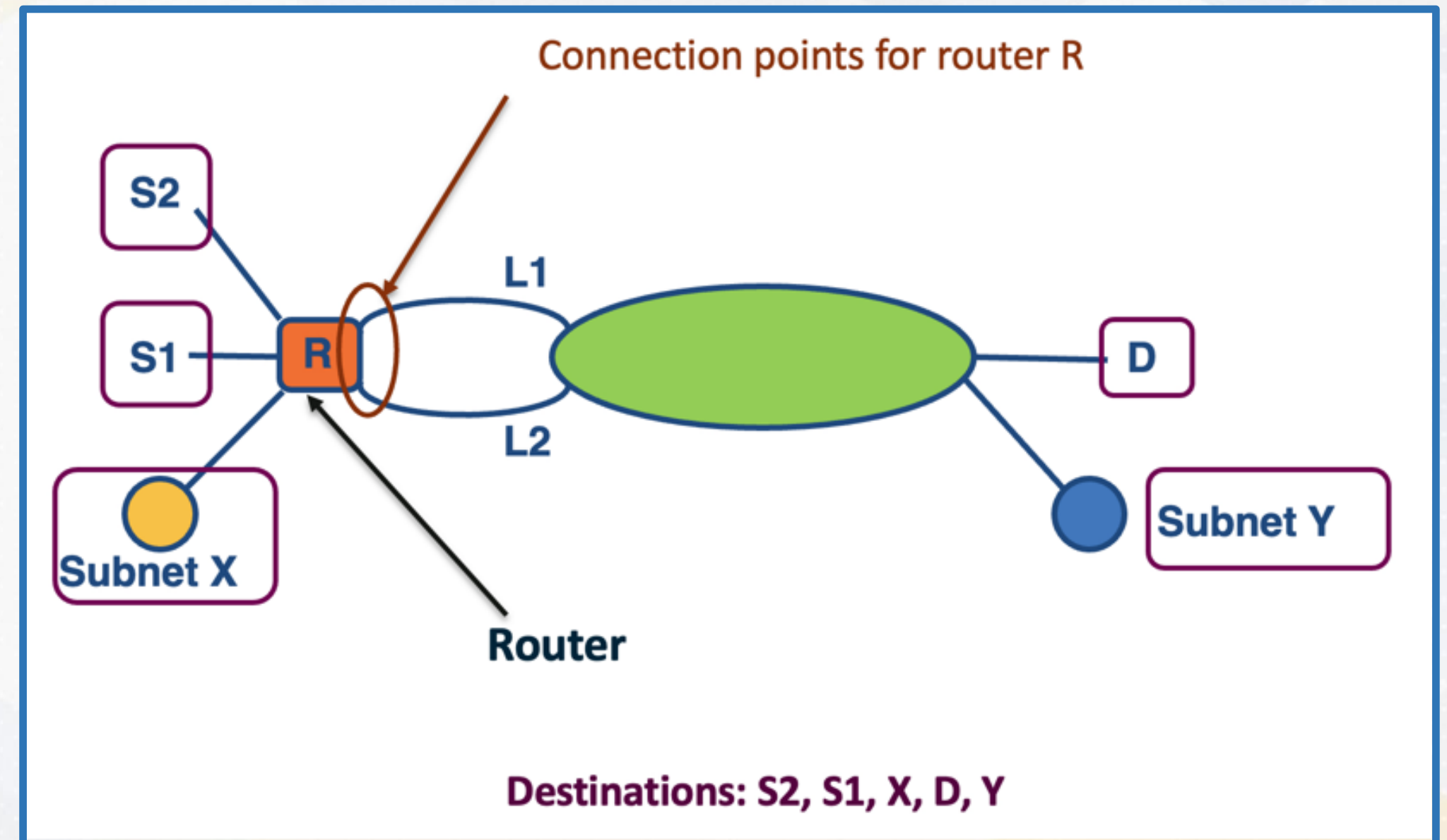
Take a ticket scheduling mechanism (3/3)



- After Round 3 continues the same loop: **request** → **ticket grant** → **connect**.
- More rounds generally mean **higher throughput** and fewer idle outputs, especially under load.
- Tradeoff: extra rounds add **scheduler work/latency**, so designs pick a small number of iterations.

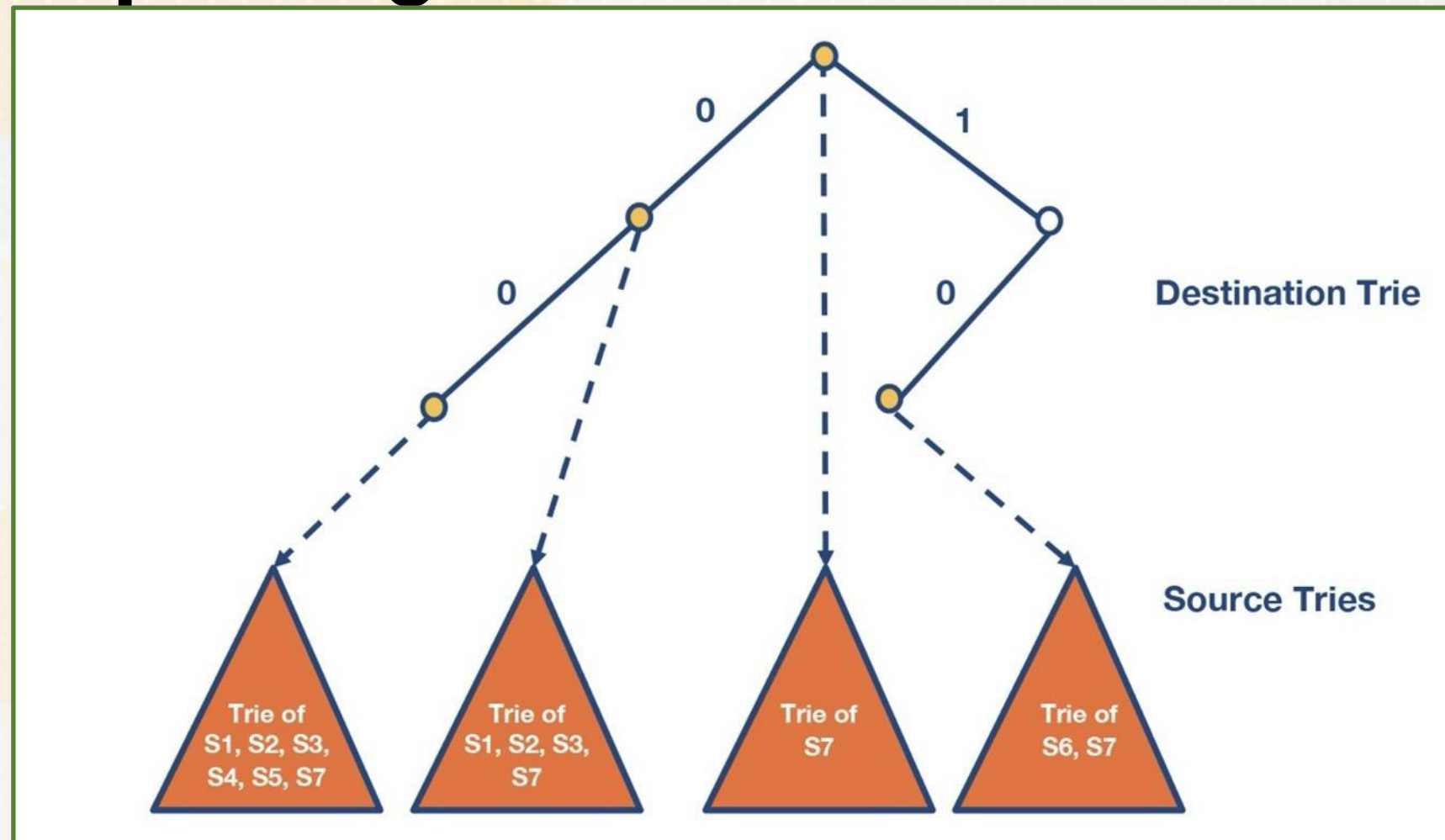
Packet Classification

- **Problem:** destination-prefix forwarding isn't enough—routers often need different actions for different *types* of traffic.
- **Goal:** assign each packet to a **policy class** (QoS, security, measurement) using multiple header fields, then apply an action.
- **What it uses:** fields like **src/dst IP, src/dst ports, TCP flags**; the router may treat traffic differently across links/destinations.



Packet Classification

Set-pruning tries



Let's assume that we have a **two-dimensional rule**. For example, we want to classify packets using both the source and the destination IP addresses.

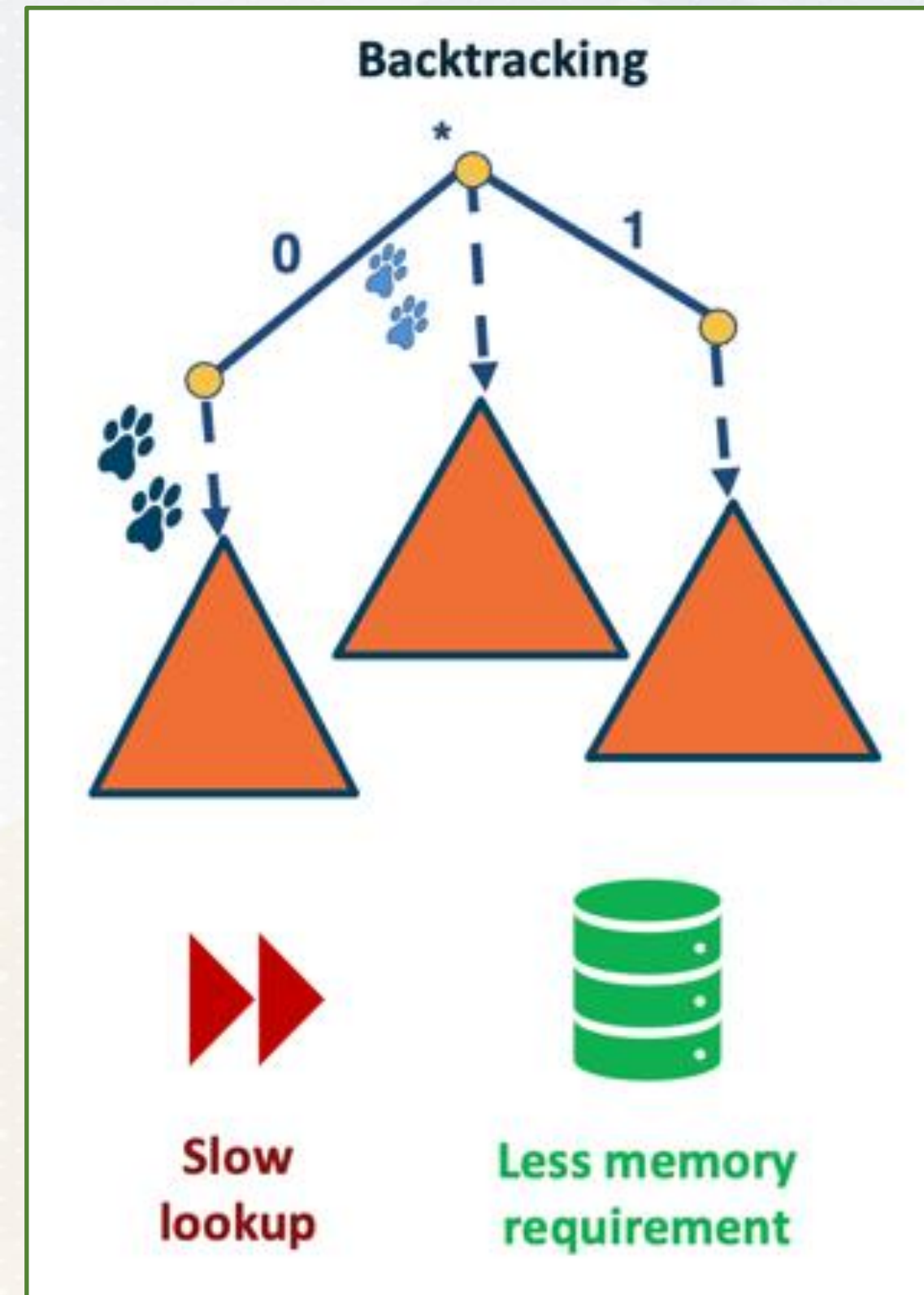
1. Build a **destination-prefix trie** first; each node represents a destination prefix region.
2. At each destination node, attach a **source-prefix trie** containing only the rules that can match within that destination region (the "candidate set").
3. **Lookup = walk destination bits** to pick the right destination node, then walk **source bits** in its attached source trie → fast, but uses more memory.

Packet Classification

Backtracking

Assume again a **two-dimensional rule**. For example, we want to classify packets using both the source and the destination IP addresses.

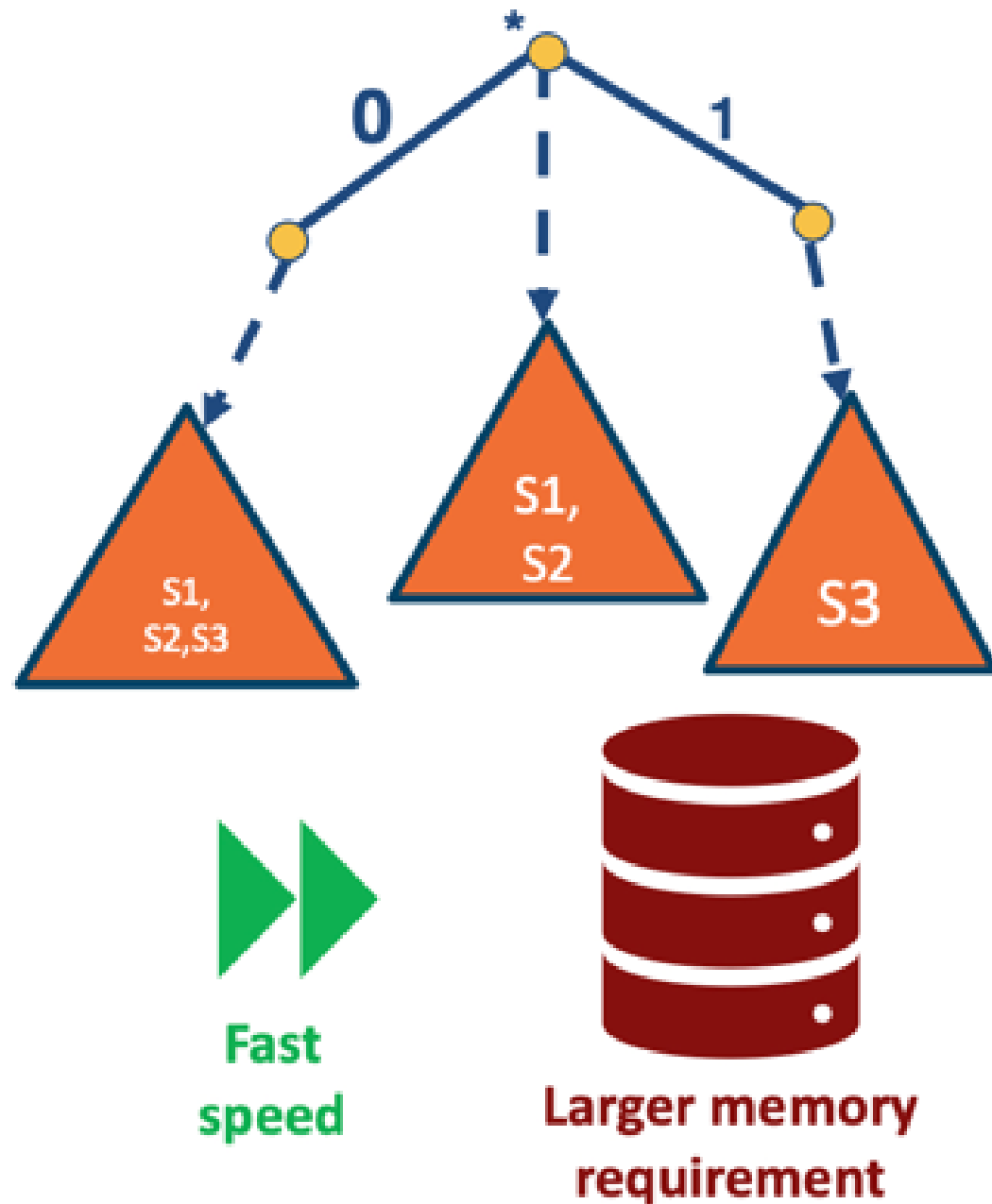
- **Idea:** avoid duplicating rule-sets across many destination nodes; store fewer source-tries → **less memory**.
- **Cost:** lookup may need **backtracking**: if the first destination-node's source-trie doesn't find a match, **move up the destination trie and try a broader node's source-trie**.
- Tradeoff: **slower lookup** (multiple tries per packet) vs **smaller memory footprint**.



Packet Classification

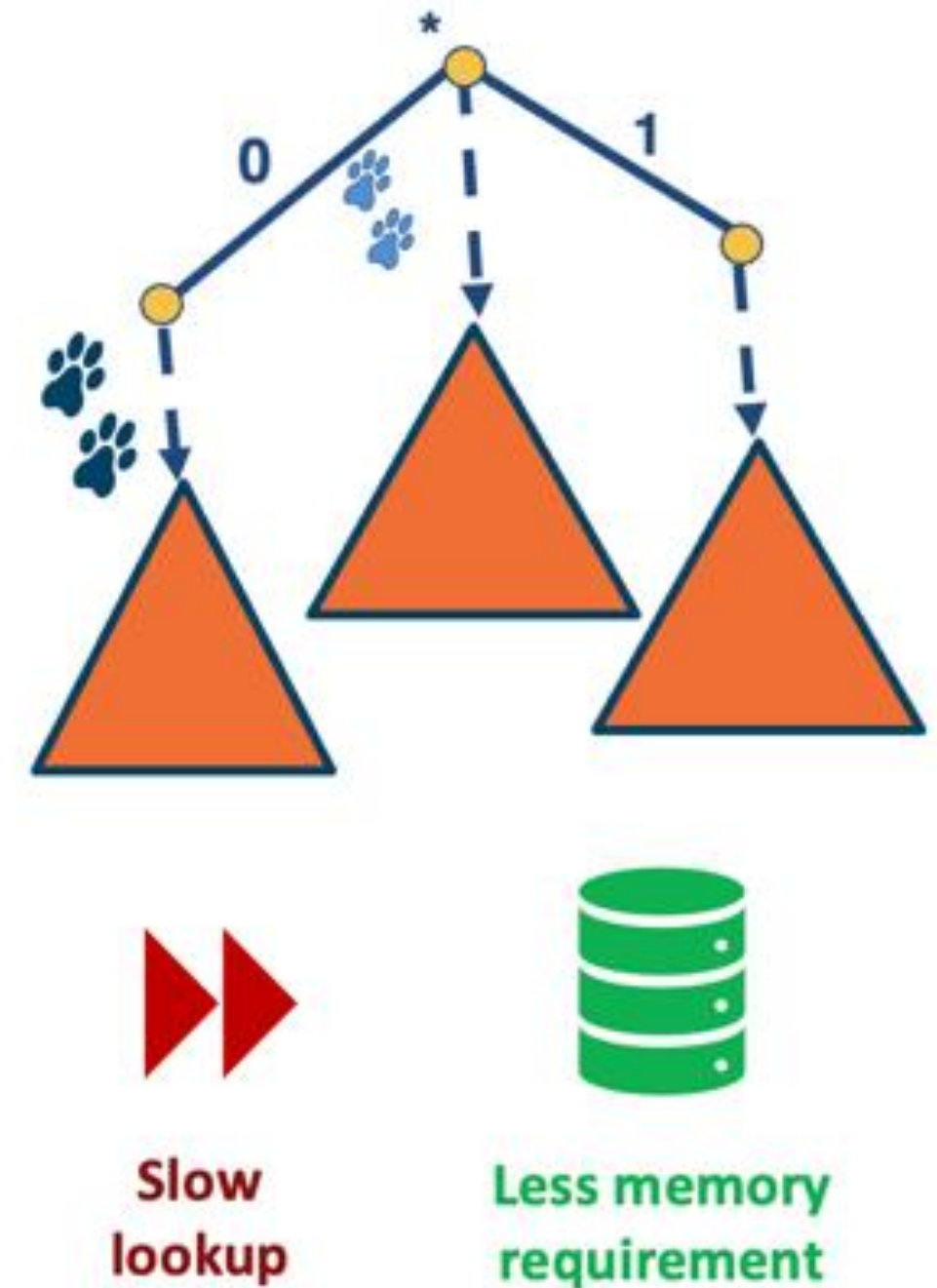
Set-pruning tries vs Backtracking

Set-pruning Tries



- **Set-pruning**: attach a pruned **source-trie per destination node** → usually **one pass** (dest walk, then source walk).
- **Backtracking**: store fewer source-tries, but may try **multiple destination levels** (dest walk + repeated source lookups).
- Core tradeoff: **speed vs memory** — set-pruning is faster but larger; backtracking is smaller but slower.

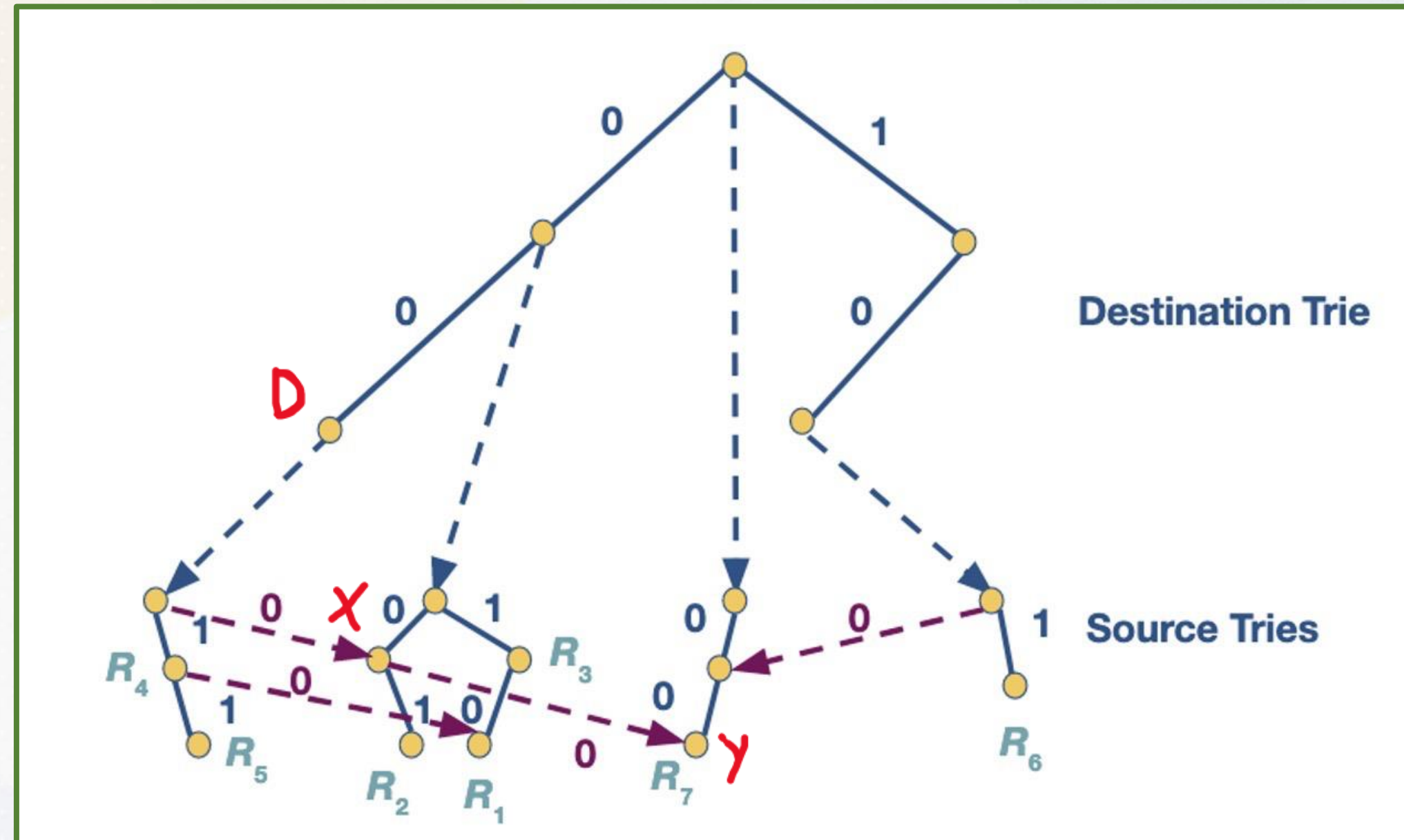
Backtracking



Packet Classification: Grid of Tries

Precomputed pointers

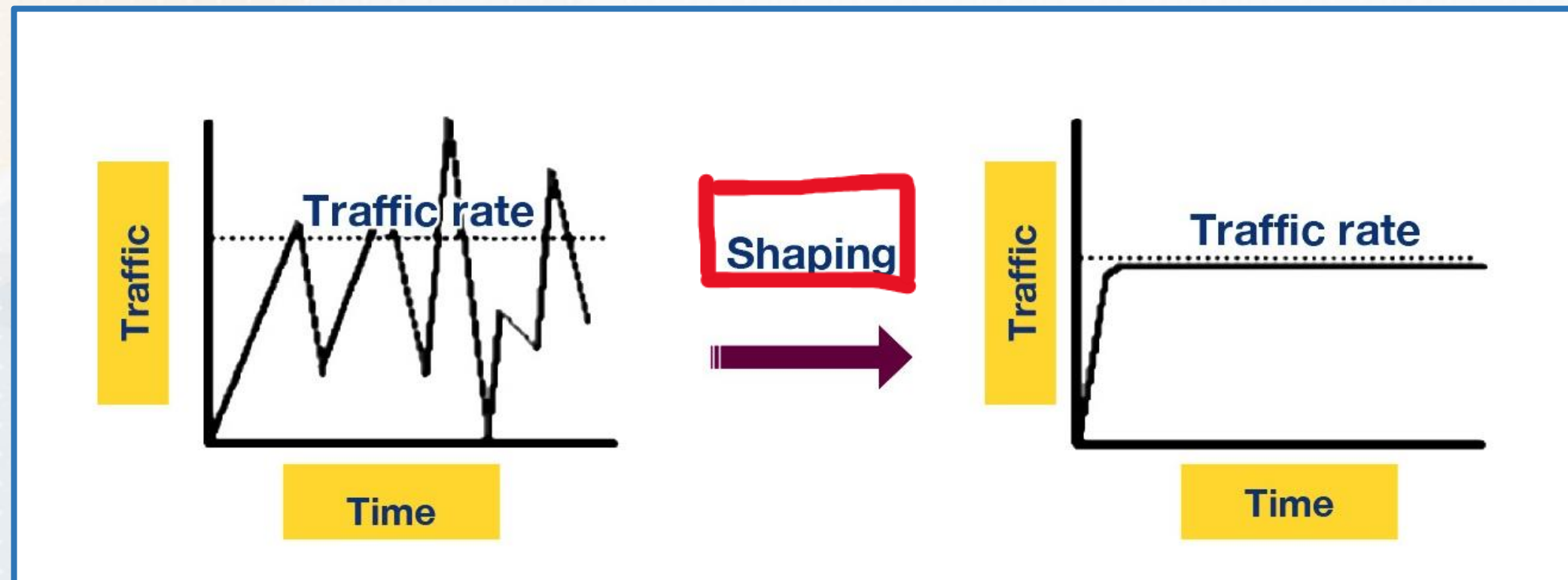
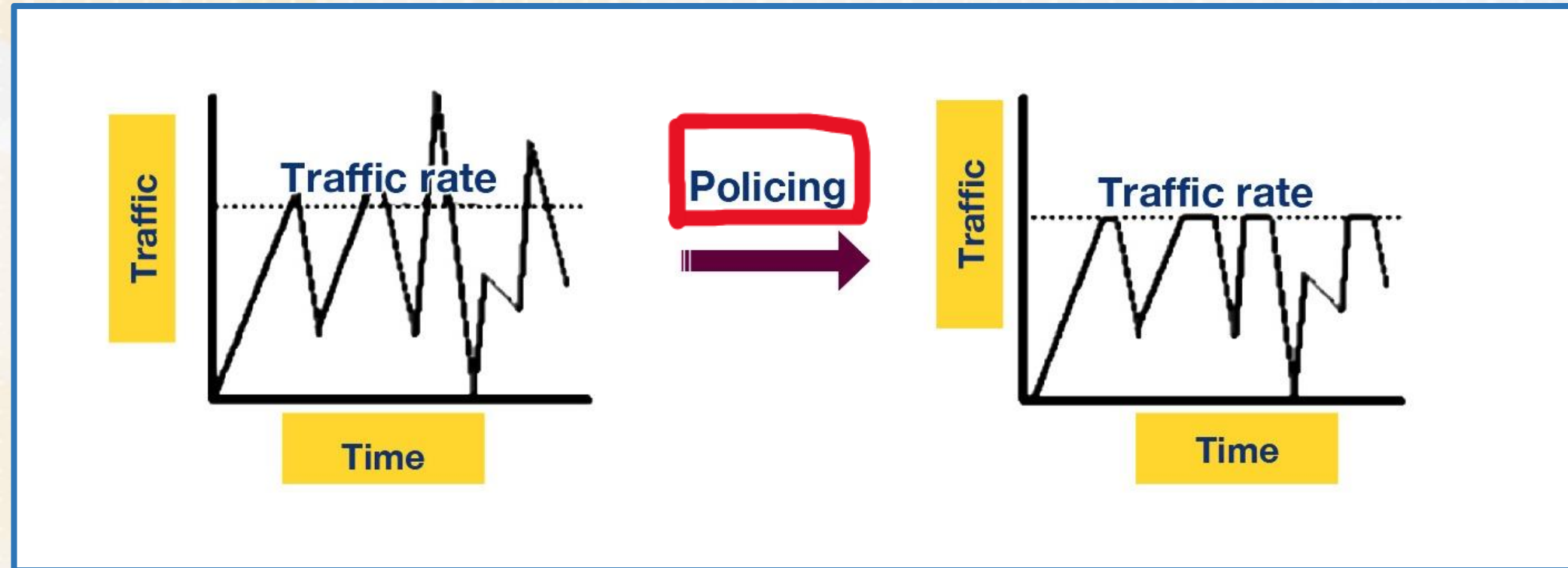
Two 2D options so far:
set-pruning (fast, heavy memory) vs **backtracking** (low memory, slow time).



- **Grid of tries:** destination trie on top; source tries underneath—but add **precomputed switch pointers** at source-trie failure points.
- Switch pointers jump to the **next relevant source trie** that could improve the match, skipping useless backtracking.

Traffic Management/Rate Control

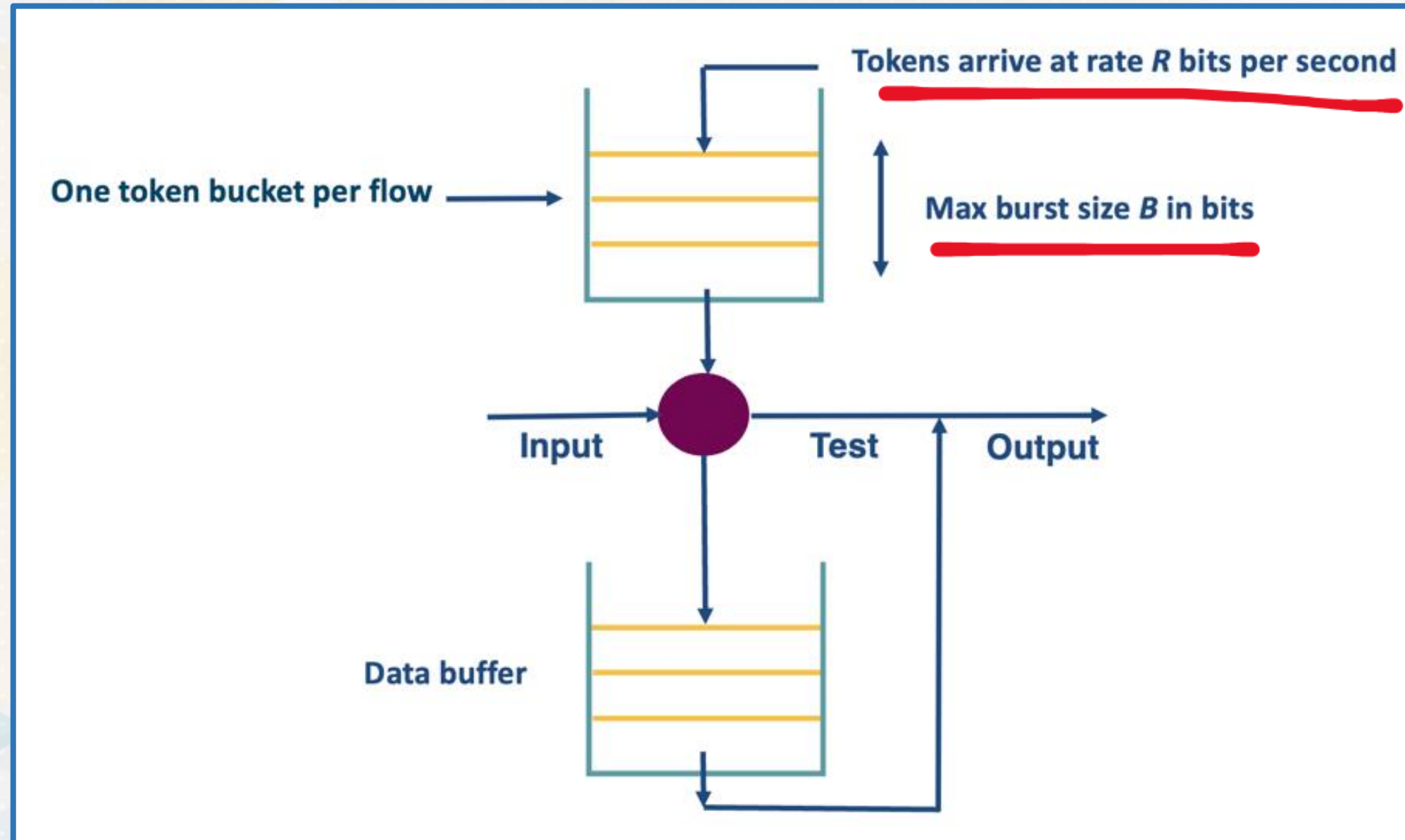
Traffic Policing vs Shaping



- **Problem:** bursts can overload queues → high delay and tail drops; we need **rate control** at the edge or in routers.
- **Shaping:** **buffer and delay** excess packets to smooth traffic to a configured rate.
- **Policing:** **drop or mark excess packets** when traffic exceeds the configured rate.

Traffic Management/Rate Control

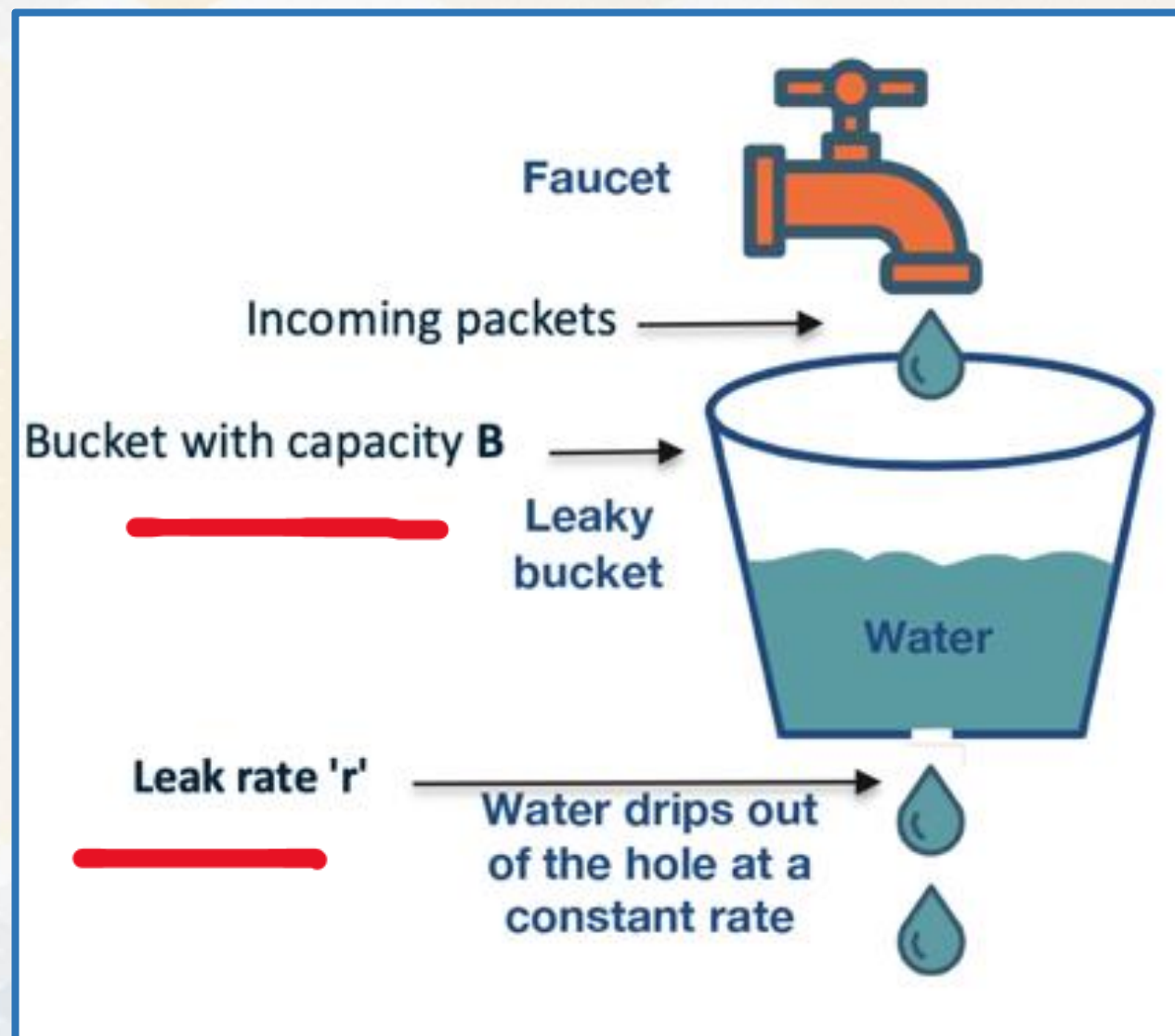
Token Bucket



- Tokens accrue at rate R up to bucket size B → controls **average rate** and **max burst**.
- A packet of size L needs L **tokens**: if tokens exist, send; if not, **wait (shaping)** or **drop/mark (policing)**.
- Allows short bursts up to B , but long-run sending rate stays near R .

Traffic Management/Rate Control

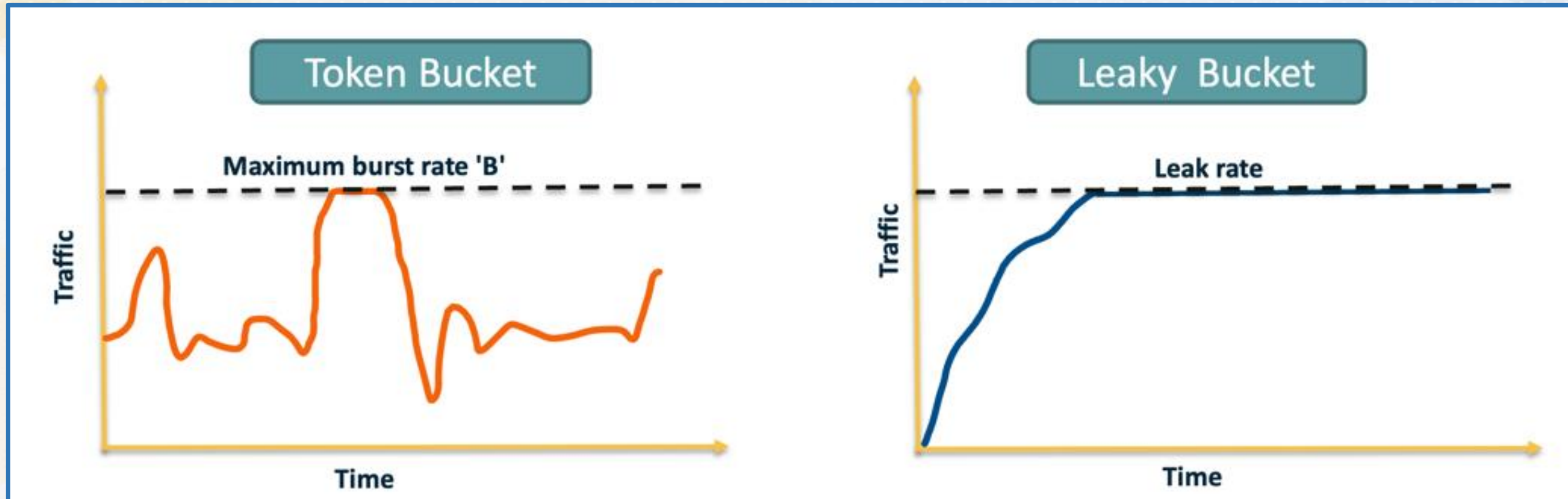
Leaky bucket



- Leaky bucket enforces a **constant output rate r** by serving from a bucket/queue at a steady pace.
- Bucket **capacity b** limits how much can be buffered; if full, excess is **dropped (policing) or delayed (shaping)**.
- Effect: **smooth traffic** and reduce burstiness seen by the network.

Traffic Management/Rate Control

Token Bucket vs Leaky Bucket



- **Token bucket**: allows bursts when tokens have accumulated; enforces **average rate R** and **burst limit B** .
- **Leaky bucket**: outputs at a **near-constant rate r** , smoothing bursts into a steady stream.
- Tradeoff: token bucket is **burst-tolerant**; leaky bucket is **smoother/stricter about output rate**.

Summary

- **Routers split into a fast data plane and a slower control plane.** Data plane forwards at line rate; control plane computes/installs the forwarding state.
- **Router forwarding is a pipeline with real bottlenecks.** Lookup → switching → queuing/scheduling → output, and contention/queues create delay and loss.
- **Fast lookup hinges on longest-prefix match and efficient data structures.** We covered **LPM**, **unibit tries**, **multibit tries**, and **prefix expansion/collisions** as speed–memory tradeoffs.
- **Crossbar switches need scheduling to avoid wasted capacity.** HOL blocking motivates **VOQs** and schedulers like **request–grant–accept** and **take-a-ticket** style arbitration.
- **Modern routers enforce policy, not just forwarding.** Packet classification enables QoS/security/measurement, and rate control uses **policing vs shaping** with **token bucket** and **leaky bucket** mechanisms.